



División de Ciencias Básicas e Ingeniería
Proyecto Terminal de ingeniería en electrónica

IoT ESP32: Red de comunicación y monitoreo mediante sensores
(MPU6050)

Autor:

José Joel Aguilar de la Cruz

Asesores:

Prof. Ricardo Marcelín Jiménez

Prof. Oscar Yáñez Suarez

Coordinador:

Prof. Michael Pascoe Chalke

Trimestre: 25 invierno, mayo 2025. Ciudad de México.

A mi familia, mamá Vianey, papá Joel y Hermano Manuel, el motor que me impulsó en esta etapa y capítulo de mi vida ...

Agradecimientos

A mis padres que como recalco con frecuencia, son el motor de mi vida, les agradezco todo su apoyo, tanto mi madre como mi padre siempre han visto por mí, sus enseñanzas y crianza fue la mejor posible y reconozco su esfuerzo. Ellos siempre trataron de que no me faltara nada, dándome lo indispensable y siempre un poco más (espero estar a la altura de merecerlo y por ello demostrarlo con cada uno de mis logros). A mi hermano Manuel le agradezco su apoyo, que pasó de ser el hermano menor, mi compañero de vida y juegos, a muchas veces ser mi consejero y apoyo en temas que no entendía durante esta última parte de la carrera (mi hermano menor me alcanzó y a veces llego a opinar sin dudar, que él es el más brillante de los dos, por momentos, en algunos campos y aspectos).

Al profesor Ricardo Marcelín Jiménez, lo conozco desde mediados del 2022 cuando tomé la uca de algoritmos y estructura de datos. Gracias por sus consejos y su paciencia, y en general por creer en mí, y que con su guía fue posible poder llegar a este momento. Gracias, por tanto, y ser un sensei en toda la extensión de la palabra (gracias por el peluche de uamito, que sin querer fue un regalo de una de sus ponencias que pude ganarme, lo atesoro como a muy pocas cosas en este mundo).

Al profesor Oscar Yáñez Suárez, que fungió como la parte más estricta del proyecto, la que recalca mis errores de manera estricta, pero con el fin de obtener el mejor resultado. Aportó en mí el creer en mí y buscar siempre ser el mejor (y gracias por siempre compartir de las galletas o chucherías que llevaba a los días de revisión).

Al Coordinador de la carrera el profesor Michael Pascoe Chalke, por su guía y atención a mis consultas sobre la carrera.

A los profesores encargados de tomarse la molestia de leer y revisar este, mi trabajo de investigación, y los que funjan como jurado de lo que aquí les presento.

A Ricardo y Diego, mis compañeros de proyecto... Cuyo apoyo le agradezco en especial a Ricardo por sus aportes a ratos en el laboratorio, reconozco que me diste una de las lecciones más grandes para no rendirme durante la última parte del año 2024, me enseñó a ser fuerte y no rendirme el verte llegar a retomar tu vida universitaria después de tu operación y estancia en el hospital, sin ti este proyecto no lo hubiese podido concluir a tiempo.

Al profesor Othon Gandarilla Carrillo por las materias cursadas remotamente con él durante los últimos 3 trimestres de mi carrera, fueron tardes arduas y de mucho aprendizaje, siempre trate de estar a la altura de sus cursos.

A José Aníbal García Gabriel por ser un amigo incomparable, a él le debo su guía y enseñanza en un sinfín de temas que muchas veces no entendía, se tomó la molestia de asesorarme y fungir como un tutor, me prestaba libros y material en momentos cruciales, y donde por instantes sentía que no podría con la carrera, y le agradezco que de esta manera nunca dejó de creer en mí y me enseñó que con honor hasta el final y nunca quitar el dedo del renglón (nunca me rendiré amigo).

A mi amigo del alma por acompañarme más de un año en mi aventura de vivir lejos de casa. David de Jesús Bautista Meléndez, cada anécdota resuena en mi mente y queda en mi corazón. Mi barra de experiencia ciertamente creció exponencialmente contigo y nuestras aventuras (series como HIMYM nos acompañaron y enseñaron durante cada comida).

Al resto de mi familia, tíos, primos, sobrinos, etcétera, y vaya que la familia es grande, tanto por parte de mi mamá, como de mi papá, y son parte esencial en mi formación, ya que, de cada una de estas personitas, fui aprendiendo y retomando lo mejor. A mi tía Elba que fue como mi segunda mamá y me cuido desde que tenía unas pocas semanas de nacido, debido a que mis padres debían de trabajar fungiendo como maestros en su escuela secundaria.

A todos mis conocidos a los que puedo llamar amigos, que me apoyaron, me dieron un consejo, su compañía o simplemente palabras de aliento para salir adelante (hasta un dibujo de borreguito en mi calendario un 3 de diciembre). Suenan en mi mente entre esos amigos; Memo, Don Salomón, Don Daniel, Don Raúl, José Andrés, Uriel, Rodolfo Chena, Ezequiel, Beto, David Vázquez, Erick, Brandon Alexis, Miguel, Saraí, Rocío, Daniel, Gabi, Yochabel, Karen, Dafne; son los algunos de los amigos que llevaré de esta etapa de mi vida en mi memoria.

Y todo aquel que por las prisas no vaya a mencionar, pero haya compartido algún momento importante de tal manera que haya generado confianza en mí y estará sí o sí en un recuerdo que atesoraré en mi memoria, o que por otro lado haya conocido después de realizar este escrito (si fue así, ten por seguro que un amigo mío lo será para toda la vida, aun estando o no en este escrito).

En general a cada persona, familiar, amistad, o conocido que me haya apoyado, aconsejado, acompañado o darme ánimo.

Le debo mis dones al mundo, y por ello soy la persona que soy en este presente. Espero seguir formando al hombre de bien que necesita este mundo y mi familia.

Abstract:

This project aimed to develop an IoT communication network using ESP32 microcontrollers and MPU6050 sensors, to create a real-time inertial monitoring system. The network consists of 5 nodes, where the central one is the ESP32-B, which coordinates communication and collects data from the peripheral nodes (ESP32-A,C,D,E), which measure movement and orientation using the gyroscope and accelerometer sensor. The ESP-NOW protocol is used for peer-to-peer communication, I2C for interface with the sensor and low consumption modes (deepsleep) to optimize energy use. The system is programmed in MicroPython, thus prioritizing accessibility and versatility. In addition, an interface with Telegram is implemented for remote data monitoring. Results include successful interconnection of devices, display of data on the OLED display, and identification of challenges and conflicts between WiFi and ESP-NOW. The project demonstrates the viability of accessible technologies in educational and research applications, with the potential to scale in areas such as structural monitoring or robotics. Finally, it is proposed to improve future results and conclusions on the results obtained, along with a section of annexes where the code with which the system works is shown.

Resumen:

Este proyecto tuvo como objetivo poder desarrollar una red de comunicación IoT mediante microcontroladores ESP32 y sensores MPU6050, para crear un sistema de monitoreo inercial en tiempo real. La red consta de 5 nodos, en donde el central es el ESP32-B, el cual coordina la comunicación y recopila datos de los nodos periféricos (ESP32-A,C,D,E), que miden movimiento y la orientación mediante el sensor de giroscopio y acelerómetro. Se emplean los protocolos ESP-NOW para la comunicación peer-to-peer, I2C para interfaz con el sensor y modos de bajo consumo (deepsleep) para optimizar el uso de la energía. El sistema se programa en MicroPython, de esta manera priorizando accesibilidad y versatilidad. Además, se implementa una interfaz con Telegram para el monitoreo remoto de datos. Los resultados incluyen la interconexión exitosa de los dispositivos, visualización de datos en la pantalla OLED y la identificación de desafíos y conflictos entre WiFi y ESP-NOW. El proyecto demuestra la viabilidad de tecnologías accesibles en aplicaciones educativas y de investigación, con potencial para escalar en ámbitos accesibles en aplicaciones educativas y de investigación, tales como entornos de monitoreo estructural o robótica. Al final del trabajo se proponen mejorar futuras y conclusiones sobre los resultados obtenidos, junto con un apartado de anexos donde se muestran los códigos con los que trabaja el sistema.

ÍNDICE GENERAL

APARTADO	PÁGINA
Dedicatoria	2
Agradecimientos	3
Resumen/Abstract	5
Índice General	6
1. Introducción	7
2. Marco Teórico	9
2.1 Microcontrolador ESP32	9
2.2 Protocolo ESP-NOW	10
2.3 Protocolo DeepSleep	11
2.4 Protocolo I ² C	12
2.5 Sensor MPU6050	13
3. Desarrollo	14
3.1 Antecedentes	15
3.2 Análisis del problema o contexto	16
3.3 Uso de MicroPython	17
4. Resultados	18
4.1 Documentación del código ProtNowABCDEv4.1_ESP32-B	19
4.2 Documentación del código ProtNowABCDEv4_ESP32-B	21
4.3 Documentación del código ProtNowABCDEv4_ESP32-ACDE	23
4.4 Documentación del código MPU6050	27
4.5 Documentación del código ssd1306	27
4.6 Conexión con el bot de Telegram	27
4.6.1 API para bots de Telegram en NodeJS	30
5. Discusión de Resultados	35
5.1 Mejora con implementación de otro sensor	35
5.2 Operación en versión local (ESP-NOW)	35
5.3 Operación con conexión a Telegram	38
6. Mejoras a Futuro y Conclusiones	40
7. Anexos	42
7.1 Código ProtNowABCDEv4.1_ESP32-B	42-50
7.2 Código ProtNowABCDEv4_ESP32-B	51-57
7.3 Código ProtNowABCDEv4_ESP32-ACDE	58-63
7.4 Código ProtNowABCDEv4_ESP32-ACDE_Telegram	63-69
7.5 Código mpu6050.py	69-71
7.6 Código ssd1306.py	71-73
7.7 Imagen de los puertos existentes del ESP32	73
8. Referencias	74-75

El internet de las cosas (IoT, por sus siglas en inglés: Internet of Things) es un gran cambio en la manera que objetos cotidianos interactúan con el mundo y entre estos mismos. Todos los sistemas de comunicación se basan en señales eléctricas o de otra naturaleza, el IoT permite que dispositivos comunes, como electrodomésticos hasta máquinas industriales se puedan conectar a internet, intercambiar datos y funcionen de manera inteligente, sin la intervención humana directa.

Para las generaciones actuales con el IoT, es algo natural: desde poder despertar con asistentes virtuales que regulan parámetros como temperatura del hogar, hasta recibir notificaciones del refrigerador de la cocina cuando falta comida. La posibilidad de controlar dispositivos, hacer compras en línea o monitorear la salud con dispositivos vestibles es parte de la vida diaria, gracias a décadas de avances tecnológicos hemos pasado de aplicaciones especializadas a soluciones accesibles para todos.

El presente trabajo forma parte de un proyecto de mayor alcance que tiene como objetivo general crear una arquitectura de IoT multipropósito. Para poder gestionarlo se ha dividido en micro proyectos a cargo de diferentes alumnos de licenciatura que pueden trabajar de manera individual o por equipos.

En este caso particular, se buscó conocer totalmente la placa ESP32, sus pines y funcionamiento de cada uno. Algo que pareciese tarea sencilla, sirvió como la antesala de todo, puesto que no se podría empezar sin antes conocer el funcionamiento y cómo se podía escalar los diferentes usos posibles.

Posteriormente la elección del lenguaje a usar (el cual fue MicroPython por su versatilidad), con ello empezaron pequeñas aplicaciones como la medición de los valores de potenciómetros, activación de LEDs, implementación de botones, y el uso de herramientas del microcontrolador más complejas como el de los pulsos PWM, y así escalonadamente hasta poder llegar al uso del sensor MPU6050 y la pantalla OLED, y las demás características que componen al proyecto

Los alcances del internet de las cosas (IoT) se extiende hacia un futuro donde la hiper conectividad transformará de forma radical nuestra interacción con el entorno, permitiendo que objetos cotidianos, infraestructura urbana y sistemas industriales, puedan operar de manera autónoma e inteligente. Con resultados que revolucionan sectores como salud, logística, infraestructura, entre otras, que generan eficiencia, sostenibilidad y nuevas oportunidades económicas. Todo esto en conjunto y cabe mencionarlo, con la inteligencia artificial, las nuevas generaciones de comunicaciones móviles 5G/6G y el Edge Computing amplifican el impacto, creando ecosistemas interconectados con capacidad de aprender, adaptarse y responder de forma predictiva a las demandas de la sociedad digital.

El presente trabajo tiene como principal objetivo desarrollar una red de comunicación IoT utilizando microcontroladores ESP32 y sensores MPU6050, con

el fin de crear un sistema de monitoreo inercial en tiempo real. El proyecto se enmarca dentro de un contexto educativo y de investigación, donde se busca demostrar la viabilidad de la utilización de tecnologías accesibles y versátiles, por mencionar; el protocolo ESP-NOW y MicroPython, las cuales tienen aplicaciones prácticas en el ámbito de la electrónica embebida.

La red mencionada consta de 5 nodos (ESP32 etiquetados de A hasta E), donde el nodo central es B (ESP32-B) el cual coordina la comunicación y recopila los datos de los sensores, mientras que los nodos periféricos (ESP32-A, ESP32-C, ESP32-D, ESP32-E) se encargan de enviar información sobre movimiento y orientación.

En las siguientes secciones, se abordan concisamente los aspectos teóricos y técnicos que sustentan el proyecto. Primeramente, se presentará el marco teórico, donde se describen las características del microcontrolador ESP32, los protocolos de comunicación utilizados (ESP-NOW, I2C y DeepSleep) y el sensor MPU6050. Posterior a esto, en la sección de desarrollo, se detallará el proceso de implementación del sistema, desde la configuración inicial de los dispositivos hasta la programación de los códigos en MicroPython. También se incluirá un análisis de los desafíos enfrentados y las soluciones adoptadas, en temas como la calibración del sensor y la gestión de la comunicación de los nodos.

Finalmente, en los apartados de resultados y discusión, se expondrán los logros alcanzados, como la interconexión exitosa de los dispositivos y la visualización de datos en una pantalla OLED, así como limitaciones identificadas, como pudiese ser los conflictos entre la conexión Wi-Fi y el protocolo ESP-NOW. Además, se propondrán mejoras futuras, dirigidas hacia la integración de otros sensores o la optimización del consumo energético.

En palabras someras este documento busca servir como referencia para futuros proyectos similares, destacando la importancia de la innovación y la accesibilidad en el desarrollo mediante la aplicación del IoT.

2. Marco Teórico

En esta sección la consideración de los temas y aspectos, es de acuerdo con la relevancia que tuvieron para la perspectiva del proyecto en IoT:

El ESP32 es un microcontrolador que fue desarrollado por Expressif Systems, diseñado para aplicaciones IoT (Internet de las cosas) y sistemas embebidos¹. Es la evolución del ESP8266, con mayor potencia, más periféricos y conectividad dual (Wi-Fi y Bluetooth).

2.1 Microcontrolador ESP32

Sus características principales:

- Procesador: Doble Núcleo Xtensa LX6 (de hasta 240 MHz)
- Memoria: 520 kb de SRAM, 4MB de Flash (según el modelo)
- Conectividad: Wi-Fi 802.11 b/g/n (2.4 GHz) + Bluetooth 4.2 (BLE y Classic).
- Periféricos: ADC, DAC, PWM, UART, SPI, I^2C , I^2S , sensores táctiles.
- Pines: 38 pines (en este caso), con múltiples usos GPIO.
- Consumo de energía configurable: Modos de bajo consumo (Deep Sleep, Light Sleep).

Ventajas	Desventajas
▪ Conectividad integrada: No requiere módulos externos para Wifi y Bluetooth. ▪ Potencia y versatilidad: Doble Núcleo y amplia gama de periféricos. ▪ Bajo coste: Accesible para proyectos educativos y profesionales. ▪ Comunidad activa: Gran cantidad de librerías y documentación	> Consumo de energía elevado en modo activo: No es ideal para aplicaciones solo a batería. > Limitaciones en GPIO: Algunos pines tienen funciones restringidas (como solo entrada, solo salida o flash) > Curva de aprendizaje para ESP-IDF: La programación requiere de más conocimiento que otras placas como Arduino, en este caso se usó micro Python (versátil y muy completo dentro de las limitaciones que tenía).

Tabla 1. Ventajas y desventajas del ESP32.

¹ Sistema embebido: “embedded system” es una combinación de hardware y software diseñado con una función específica a cumplir.

La utilización del ESP32 en redes de comunicación IoT (Internet de las cosas) se ha consolidado debido a que es una solución accesible:

- Conectividad integrada
- Bajo consumo energético
- Compatibilidad con sensores como el MPU6050: vía I2C (Espressif Systems, 2023)

2.2 Protocolo ESP-NOW

ESP-NOW es un protocolo de comunicación inalámbrica desarrollado por Espressif Systems para los dispositivos basados en ESP32 y ESP8266. El cual permite la transmisión de datos rápida, a baja latencia y sin conexión de cables (connectionless) entre dispositivos, sin necesidad de Wi-Fi tradicional o infraestructura de red.

Algunas de sus características principales son:

- Conexión peer-to-peer (P2P): Comunicación sin router
- Baja latencia: Ideal en aplicaciones en tiempo real
- Encriptado opcional: Soporta AES-128 ² para seguridad.
- Sin handshaking: Reduce overhead ³ en la comunicación
- Alcance similar a Wi-Fi: Depende del entorno (hasta ~200m en espacio abierto).

Ventajas	Desventajas
▪ Conectividad integrada: En comparación del Wi-Fi estándar. ▪ Configuración sencilla: No requiere direcciones IP. ▪ Compatibilidad ESP: ESP32 Y ESP8266	> Limitado a dispositivos Espressif > Sin retransmisión automática > Menor alcance que Lora o Zigbee

Tabla 2. Ventajas y desventajas del protocolo ESP-NOW

Aplicaciones típicas:

- > Redes de sensores inalámbricas: ejemplo; monitoreo ambiental.
- > Control remoto de dispositivos: robots, drones.
- > Sistemas domóticos: iluminación y cerraduras inteligentes.
- > Comunicación máquina a máquina: M2M.

2.3 Protocolo DeepSleep

² AES-128 es una de las tres variantes del estándar AES (junto con AES-192 y AES-256), que emplea claves de 128 bits para cifrado. National Institute of Standards and Technology (NIST). (2001). *Advanced Encryption Standard (AES)* (FIPS PUB 197). U.S. Department of Commerce. <https://doi.org/10.6028/NIST.FIPS.197>

³ Overhead: (en español “sobrecarga” o “sobrecoste”) es referido a un consumo adicional de recursos (tiempo, ancho de banda, memoria o energía) que un sistema utiliza para gestionar sus operaciones, por encima de lo estrictamente necesario para realizar la tarea principal.

En el modo “Deepsleep”, el ESP32 apaga la CPU y el oscilador de cristal correspondiente. Un oscilador de cristal asegura que el tiempo siga corriendo. Dicho oscilador interno es muy impreciso y depende de la temperatura, lo que ocasiona que en el modo “deepsleep” se pueda producir rápidamente una desviación (o deriva) del 20% o más. Lo que puede causar que después de unos días el reloj se desincronice.

Asimismo, la hora se puede reajustar a 0 durante cambios de batería, un reset o un watchdog. Esto es bastante problemático para el funcionamiento con batería de larga duración; esta restricción existe prácticamente en todas las tarjetas ESP32⁴.

Al entrar en modo Deep sleep, en algún momento se deberá regresar al modo “Normal”. Para esto se puede disponer de varias formas de “despertar” el ESP32. A los cuales se les denomina Wake-Up Sources.

Características principales:

- ❖ Consumo ultra bajo: entre 5 μ A y 150 μ A dependiendo la configuración.
- ❖ Retención de memoria: Solo la memoria RTC (ULP) y registros críticos
- ❖ Fuentes de wake-up (despertar):
 - o Timer (RTC): activación tras un tiempo programado.
 - o GPIO externo: interrupción por pulsador, sensor, etc.
 - o Touchpad: Sensores táctiles (en algunos modelos)
- ❖ Reinicio parcial: Al despertar el código vuelve a ejecutarse desde el inicio.

Aplicaciones frecuentes en:

- > Dispositivos IOT con baterías: Sensores Remotos (temperatura, humedad) que envían datos periódicamente.
- > Dispositivos vestibles: Pulseras inteligentes que monitorean actividad con autonomía prolongada.
- > Agricultura inteligente: Como en estaciones meteorológicas autónomas.

Modos de operación

El ESP32 nos da algunos modos de operación que influyen en su consumo energético.

- Deep sleep (sueño profundo):
 - o Consumo bajo: $\sim 5 \mu A$
 - o Componentes activos: RTC (Real-Time Clock) y memoria RTC
 - o Wake-UP: Timer, pines externos (Ext0/Ext1), touchpath.
 - o Reinicio: Tras despertar, el sistema se reinicia desde el inicio del código.
 - o Retardo del despertar: 250 ms

- Lite sleep (sueño ligero):
 - o Consumo Bajo: $\sim 0.8 mA$

⁴ <https://www.radioshuttle.de/es/medias-es/informaciones-tecnicas/esp32-alimentado-por-bateria/>

- o Componentes activos: CPU pausado, pero periféricos como WIFI pueden mantenerse activos.
 - o Wake Up: Timer, interrupciones GPIO, UART.
 - o Continúa ejecución: tras despertar, el código retoma desde donde se pausó.
- Sleep (Delay Estándar):
 - o Consumo: Alto (similar al modo activo)
 - o Uso: time.sleep() solo bloquea la ejecución, no reduce el consumo de energía.

Los modos Sleep son una estrategia usada para minimizar el consumo de energía en dispositivos electrónicos cuando no se está activamente realizando tareas.

Elementos que consumen energía:

- Frecuencia del reloj
- Periféricos y comunicaciones: por ejemplo, WIFI y Bluetooth.

Modo	Consumo	Wake Up	Reinicio	Aplicación
Deep Sleep	~ 5 μA 150 μA	Timer, Ext0/Ext1, touch	SI	Ahorro máximo de energía
Lite Sleep	~ 0.8 mA – 1.5 mA	Timer, GPIO, UART	NO	Tareas periódicas rápidas
Sleep	~ 20 mA - 100 mA	N/A (Solo hay delay)	NO	Esperas no críticas
Normal WIFI/Bt activos	~ 50 mA - 240 mA	N/A	N/A	Uso normal
Normal WIFI/Bt off (solo CPU)	~ 20 mA - 40 mA	N/A	N/A	Uso normal

Tabla 3. Características de modos de vigilia/sueño

2.4 Protocolo I2C

I^2C (Inter-Integrated Circuit) es un protocolo de comunicación síncrono, bidireccional y multimaestro que permite la transferencia de datos entre dispositivos utilizando 2 líneas:

- SDA (Serial Data): Transmisión de datos
- SCL (Serial Clock): Sincronización.

Algunas de sus características principales son:

- Velocidad: Hasta 1 MHz (en modo rápido-plus).
- Direccionamiento: 7 u 8 (hasta 127 dispositivos en el mismo bus)
- Resistencias de Pull-up: Requeridos (~ 4.7 K Ω).
- Multi-maestro: Soporta comunicación entre múltiples dispositivos controladores.

Aplicaciones típicas:

- > Sensores (BMP280, MPU6050)
- > Pantallas OLED
- > Memorias EEPROM

Ventajas	Desventajas
▪ Conexión simple: 2 cables ▪ Multi-Dispositivo: un solo bus	> Velocidad Limitada: vs SPI > Sensible a ruido eléctrico: cables largos

Tabla 4. Ventajas y desventajas del protocolo I2C

2.5 Sensor MPU6050

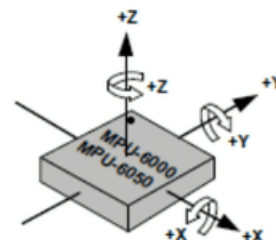
El MPU6050 es un sensor de 6 ejes fabricado por InvenSense (TDK) que combina un giroscopio (3 ejes) y un acelerómetro (3 ejes), con un procesador de movimiento digital (DMP) para aplicaciones de medición inercial.

Especificaciones y características del MPU 6050	Pines y Funciones
❖ Voltaje de funcionamiento: 3.3V a 5V ❖ Chip: MPU-6050. ❖ Grados de libertad (DoF): 6 ❖ Protocolo de comunicación: I2C ❖ Convertidor analógico a digital: 16 bit en todos los ejes. ❖ Rango Giroscopio: Rango de $\pm 250^\circ/\text{s}$ a $\pm 2000^\circ/\text{s}$. [$^\circ/\text{s}$] (dps) Acelerómetro: Rango de $\pm 2g$ a $\pm 16g$. dps: degrees per second (grados por segundo). g: aceleración de la gravedad. 9.8 m/s^2	1. VCC: Voltage Common Collector - Alimentación del módulo. 2. GND: Ground - Conexión a tierra. 3. SCL: Serial Clock Line - Línea de reloj del bus I2C. 4. SDA: Serial Data Line - Línea de datos del bus I2C. 5. XDA: Auxiliary Data Line - Línea de datos auxiliar (usualmente no se usa). 6. XCL: Auxiliary Clock Line - Línea de reloj auxiliar (usualmente no se usa). 7. AD0: Address 0 - Dirección I2C (puede ser conectada a GND o VCC para cambiar la dirección I2C del dispositivo). 8. INT: Interrupt - Pin de interrupción (señal de interrupción generada por el MPU6050).

Tabla 5. Datos del módulo MPU6050

Para la interpretación de la aceleración del MPU6050 está referida por la relación de la salida del sensor y la aceleración física que está midiendo, a esta sensibilidad se le mide con las unidades LSB/g (Least Significant Bits/gravedad).

En el MPU6050 la sensibilidad depende del rango de aceleración que se configure, alguno de esos rangos es:



- $\pm 2g$: 16384 LSB/g
- $\pm 4g$: 8192 LSB/g
- $\pm 8g$: 4096 LSB/g
- $\pm 16g$: 2048 LSB/g

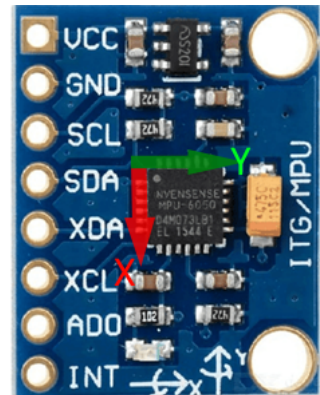
Para la interpretación de las mediciones del acelerómetro, se usó el rango de $\pm 2g$, donde cada g de aceleración se representa por 16,384 unidades en la salida del sensor. Por ejemplo, si el sensor está midiendo 1g de aceleración en el eje X, el valor crudo obtenido será aproximadamente 16,384.

Para convertir los valores de aceleración del MPU6050 crudos a unidades de gravedad (g), se necesita dividir entre la sensibilidad del acelerómetro.

La sensibilidad ocupada fue 16384 LSB/g, por lo cual los valores medidos por el acelerómetro se dividirán entre la sensibilidad 16384.

Aplicaciones típicas del sensor:

- > **Robótica:** estabilización de drones y robots.
- > **Dispositivos vestibles:** Monitoreo de movimiento en dispositivos portátiles.
- > **Realidad Virtual:** Seguimiento y orientación.
- > **Automóviles:** Sistemas de detección de inclinación.



Ventajas	Desventajas
▪ Integración de múltiples sensores: en un solo chip ▪ Bajo costo: hay en amplia disponibilidad ▪ DMP: para procesamiento interno de datos.	> Requiere calibración: para la reducción de deriva (drift) > Sensible a vibraciones mecánicas

Tabla 6. Ventajas y desventajas del sensor MPU6050.

3.- Desarrollo

Esta sección expone los fundamentos técnicos y las decisiones clave que sustentan el proyecto, el cual es integrado por 3 principales ejes: los antecedentes tecnológicos del MPU6050 y MicroPython en aplicaciones similares, el análisis de necesidades y limitaciones que justifican el uso del ESP32, y una evaluación comparativa de los componentes seleccionados.

A raíz del presente marco, se busca que la combinación ESP32+MPU6050+MicroPython responda tanto a requisitos de conectividad y accesibilidad educativa, a su vez permitiendo escalar el sistema hacia aplicaciones prácticas, como lo es el objetivo principal, con el monitoreo estructural.

3.1 Antecedentes

Los sistemas de monitoreo inercial han evolucionado desde complejos mecanismos hasta sensores MEMS (Micro-Electro-Mechanical-Systems), como el MPU6050, el cual permite mediciones precisas de aceleración y rotación en dispositivos compactos y de bajo costo. Su utilización masiva en drones, dispositivos vestibles y robótica ha democratizado el acceso a tecnologías de navegación autónoma.

Aplicaciones previas del MPU6050 en proyectos

- Robótica Educativa: En plataformas como Arduino y ESP32 lo agregan para estabilizar robots (Premarlani & Bizard, 2012).
- Salud: Monitoreo de movimiento en prótesis inteligentes (STMicroelectronics, 2020)
- Agricultura de precisión: Detección de inclinación en maquinaria autónoma.

Programación con Micro Python en sistemas embebidos

La facilidad de programación que aporta micro Python en este tipo de sistemas es debido a su sintaxis sencilla y compatibilidad con ESP32. Ejemplos como:

- Sensores inerciales (Zhang, 2022)
- Proyectos académicos: priorizando la accesibilidad sobre el rendimiento (Sánchez, 2020).

3.2 Análisis del problema o contexto

El proyecto que aquí se reporta plantea las siguientes necesidades:

- 1 Aprender a utilizar el ESP32 y su programación en MicroPython, conociendo y explorando funcionalidades de sus pines GPIO, protocolos de comunicación (I2C, SPI) y modos de bajo consumo.
- 2 Crear un sistema interconectado para monitoreo inercial, que fuese económico, con la capacidad de recopilar datos de movimiento y orientación en tiempo real, con aplicaciones en el análisis estructural de edificios.

El proyecto se enmarca en un entorno educativo y de investigación, que puede entenderse como una prueba de concepto o prototipo:

- Docencia: Enseñanza de sistemas embebidos, IoT y sensores.
- Investigación: Monitoreo de vibraciones y desplazamiento en estructuras civiles (edificios de la unidad, como es el objetivo), utilizando los datos inerciales para detectar anomalías.

Limitaciones de otras plataformas que finalmente no se usaron para el proyecto:

- ❖ Arduino UNO: Carece de conectividad inalámbrica (Wi-Fi/Bluetooth), lo que limitaba su uso en aplicaciones IoT.
- ❖ Plataformas más costosas: Raspberry + IMU industriales, debido a precios elevados.
- ❖ Sensores inerciales avanzados: como el BNO055, mayor precisión, pero con costos elevados para estos fines académicos.

La oportunidad de integración: ESP32 + MPU6050 + MicroPython

1. ESP32:
 - Conectividad integrada
 - Bajo costo
 - Compatibilidad con MicroPython
2. MPU6050
 - Costo accesible
 - Precisión suficiente
3. MicroPython:
 - Sintaxis simplificada
 - Librerías disponibles

Componente	Ventaja Técnica	Relevancia para el Proyecto
ESP32	Wi-Fi/BLE, doble núcleo, bajo consumo.	Transmisión inalámbrica de datos y procesamiento local.
MPU6050	Sensor inercial de 6 ejes con DMP integrado.	Medición de vibraciones y orientación en estructuras.
MicroPython	Lenguaje amigable y rápido para prototipado.	Ideal para entornos educativos y pruebas iniciales.

Tabla 7. Resumen sobre la justificación de los componentes.

3.3.3 Uso de Micro Python

La elección de este lenguaje fue debido a 2 principales razones:

- Sintaxis sencilla:
- Comunidad activa:

Ventajas vistas:

- Lectura rápida: Ejemplo: `mpu.get_values()` devuelve datos en milisegundos.
- Flexibilidad: Cambios en tiempo real sin recompilar.
- Agilidad: Implementación de menús y estados con pocas líneas de código.

4. Resultados

Los resultados obtenidos fueron la red de comunicación con 5 nodos denominados: ESP32-A, ESP32-B, ESP32-C, ESP32-D, ESP32-E.

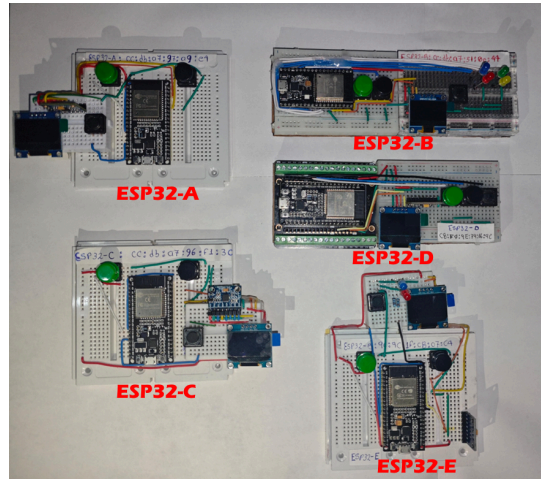


Figura 3. Red de ESP32 armada y montada en protoboards.

La red tiene como concentrador o nodo central al ESP32-B, que también sirve para controlar las demás, con 3 opciones de operación en su menú:

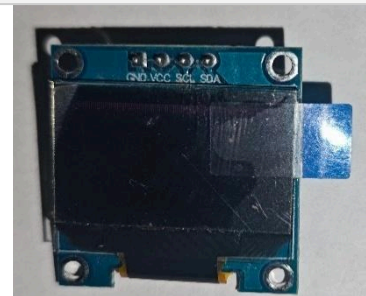
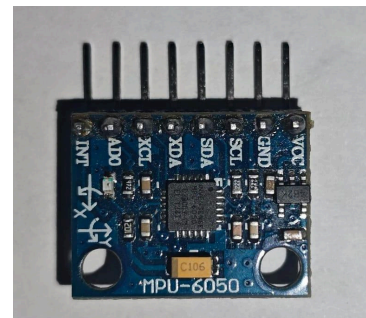
1. Ver mensajes
2. Enviar mensaje
3. Dormir

Mientras que las demás ESP32 (A, C, D, E) tienen la función de sensores que se encargan de enviar sus mediciones al ESP32-B (es decir, el nodocentral). A su vez, este conjunto tiene la capacidad de enviar mensajes individuales (de un apartado con 4 mensajes predefinidos y que se pueden editar o aumentar) hacia el ESP32 central.

Usando como periféricos, una pantalla OLED de 128x64 pixeles para visualización de datos y el sensor de giroscopio y acelerómetro MPU6050.

Los demás materiales son botones, LEDs, resistencias y cableado (calibre 24), lo que complementa el armado total de los circuitos.

La parte del software se presenta a continuación con la explicación de los códigos en MicroPython.



4.1 Documentación del código “”ProtNowABCDEv4.1_ESP32-B ”

El código implementa un sistema de comunicación entre dispositivos ESP32 utilizando el protocolo ESP-NOW para intercambiar información recabada por sensores MPU6050, así como mensajes predefinidos. Funciona como un nodo central (ESP32-B) que recibe datos de otros dispositivos (A, C, D, E), los muestra en una pantalla OLED, y los envía a un servidor de Telegram para monitoreo remoto. Además, incluye un menú interactivo con botones para navegar entre estados como enviar mensajes, visualizar datos de sensores, o entrar en modo deep sleep para ahorrar energía. El sistema también utiliza LEDs para indicar mensajes recibidos y maneja errores mediante notificaciones en Telegram

Apartados del código: ProtNowABCDEv4.1_ESP32-B (véase para mayor referencia el apartado de Anexos, de la página 42 a la 50, donde está este código en formato de colores y comentado) .

Librerías utilizadas

Importación de módulos necesarios para el funcionamiento del sistema, incluyendo control de hardware (GPIO, I2C, deep sleep), comunicación WIFI, protocolo ESP-NOW, pantalla OLED, manejo de tiempo, y conexión con Telegram.

Configuración OLED

Inicialización del bus I2C y la pantalla OLED (128x64 píxeles) utilizando los pines GPIO 22 (SCL) y 21 (SDA). Permite mostrar texto y datos en la pantalla.

Configuración de LEDs

Define los pines GPIO (16-19) como salidas para controlar LEDs que indican mensajes recibidos de cada dispositivo (A, C, D, E). Cada LED se asocia a un dispositivo específico.

Configuración WIFI y Telegram

Almacena las credenciales WIFI (SSID y contraseña) y los datos del bot de Telegram (token y chat ID). Estos se usan para conectar a Internet y enviar notificaciones.

Clase para manejo de botones

Implementa una clase Button con lógica de debounce para evitar rebotes. Gestiona tres botones físicos (menu, OK, select) conectados a los pines GPIO 12, 13 y 14.

Configuración ESP-NOW

Configura el protocolo ESP-NOW para comunicación peer-to-peer entre dispositivos ESP32. Agrega las direcciones MAC de los dispositivos pares (A, C, D, E) y activa la interfaz.

Estados del sistema

Define los posibles estados de la máquina de estados (monitor, menú, envío de mensajes, deep sleep, etc.) y establece el estado inicial (STATE_MONITOR).

Variables de control

Variables globales para gestionar opciones seleccionadas, historial de mensajes, destino de envío, datos de sensores, y estado de la conexión WIFI.

Mensajes predefinidos

Lista de mensajes predefinidos que pueden enviarse a otros dispositivos a través de ESP-NOW (ej.: "Te copio, mensaje recibido").

Funciones de conexión WIFI y Telegram

- connect_wifi(): Intenta conectar a la red WIFI y actualiza el estado global.
- send_telegram_message(): Envía mensajes al chat de Telegram configurado, con manejo de errores y desconexión automática.

Funciones de visualización

Grupo de funciones para controlar la pantalla OLED y LEDs:

- blink_led(): Parpadeo breve de un LED.
- show_text(): Muestra hasta 4 líneas de texto en la OLED.
- Funciones específicas para cada estado (show_monitor(), show_menu(), etc.).

Configuración de wakeup y inicialización

Prepara el pin GPIO 12 como fuente de wakeup para salir del modo deep sleep. Inicia la conexión WIFI y envía un mensaje de inicio a Telegram.

Bucle principal

Máquina de estados que:

1. Recibe mensajes vía ESP-NOW (normales o de sensores).
2. Gestiona interacciones con botones.
3. Actualiza la pantalla según el estado actual.
4. Maneja errores reiniciando el sistema si es necesario.

Cada apartado está diseñado para ser modular, facilitando su mantenimiento y escalabilidad. De este código existe otra versión, en la cual cambia no envía datos al servidor en Telegram y sólo se intercomunican localmente.

4.2 Documentación del código “ProtNowABCDev4_ESP32-B”

ESP32 utilizando el protocolo ESP-NOW, donde el nodo central (ESP32-B) recibe y envía mensajes a otros cuatro dispositivos (A, C, D, E). El sistema muestra los mensajes recibidos y datos de sensores MPU6050 en una pantalla OLED, utilizando LEDs para indicar el origen de los mensajes. Incluye un menú interactivo con botones para navegar entre funciones como enviar mensajes predefinidos, visualizar datos de sensores (como aceleración en ejes X, Y, Z) o activar el modo deep sleep para ahorro de energía. *A diferencia de la versión anterior, este código no depende de conexión WIFI, ni Telegram, centrándose únicamente en la comunicación local entre dispositivos.* El sistema es robusto, con manejo de errores que reinicia el dispositivo en caso de fallos.

Apartados del código: ProtNowABCDev4_ESP32-B (véase para mayor referencia el apartado de Anexos, de la página 51 a la 57, donde está este código en formato de colores y comentado).

Librerías utilizadas

Descripción:

Conjunto de módulos esenciales para el funcionamiento del sistema:

- machine: Control de hardware (GPIO, I2C, deep sleep, RTC).
- network: Gestión de conexiones WiFi.
- espnow: Protocolo para comunicación peer-to-peer entre ESP32.
- ssd1306: Control de pantallas OLED.
- time, ubinascii, esp32: Funciones auxiliares (tiempo, conversión binaria, características ESP32).

Configuración OLED

Descripción:

Inicializa la comunicación I2C (pines 21-SDA y 22-SCL) y la pantalla OLED de 128x64 píxeles. Permite mostrar texto y datos en la pantalla.

Configuración de LEDs

Descripción:

Asigna pines GPIO (16-19) a LEDs que indican mensajes recibidos de cada dispositivo (A, C, D, E). Cada LED se activa cuando llega un mensaje del dispositivo correspondiente.

Clase para manejo de botones

Descripción:

Implementa una clase Button con:

- Debounce: Evita falsas pulsaciones (tiempo de espera: 350 ms).

- Botones físicos:
 - o btn_menu (GPIO 13): Entra/sale del menú.
 - o btn_ok (GPIO 12): Confirma acciones.
 - o btn_select (GPIO 14): Navega entre opciones.

Configuración ESP-NOW

Descripción:

Configura el protocolo ESP-NOW para comunicación directa entre ESP32:

- Direcciones MAC: De los dispositivos pares (A, C, D, E).
- Modo estación WIFI: Habilita la interfaz para ESP-NOW.
- Peers añadidos: Permite enviar/recibir mensajes de los dispositivos registrados.

Estados del sistema

Máquina de estados que define los modos de operación:

- STATE_MONITOR: Muestra mensajes entrantes (estado inicial).
- STATE_MENU: Menú principal con opciones.
- STATE_SEND_TARGET/STATE_SEND_MSG: Envío de mensajes a dispositivos.
- STATE_SLEEP_CONFIRM: Confirmación para deep sleep.
- STATE_SENSOR_DISPLAY: Visualización de datos de sensores.

Variables de control.

Variables globales clave:

- current_option: Índice de la opción seleccionada en menús.
- last_messages: Historial de mensajes recibidos (máx. 3).
- selected_target: Dispositivo destino para enviar mensajes (0=A, 1=C, etc.).
- sensor_data: Almacena datos de sensores recibidos (formato S:origen:datos).

Mensajes predefinidos

Descripción:

Lista de mensajes disponibles para enviar a otros dispositivos via ESP-NOW:

Funciones de visualización

Descripción:

Controlan la interfaz de usuario:

- LEDs: blink_led() parpadea el LED del remitente.
- OLED:
 - o show_text(): Muestra hasta 4 líneas de texto.
 - o Funciones específicas (show_monitor(), show_menu(), etc.) para cada estado.
- Datos de sensores: show_sensor_data() parsea y muestra valores de acelerómetro (ej. AcX: 0.12g).

Configuración de wakeup e inicialización

Descripción:

- Wakeup: Configura el pin GPIO 12 para despertar el dispositivo desde deep sleep.
- Inicialización: Muestra un mensaje de inicio en la OLED ("Sistema iniciado...").

Bucle principal

Descripción:

Máquina de estados que ejecuta:

1. Recepción de mensajes:
 - o Detecta si son mensajes normales o datos de sensores.
 - o Actualiza la pantalla y LEDs según el remitente.
2. Gestión de botones:
 - o Navegación en menús (btn_select).
 - o Confirmación de acciones (btn_ok).
 - o Transiciones entre estados (btn_menu).
3. Manejo de errores: Reinicia el sistema si ocurre una excepción.

Este código también pertenece al nodo central de comunicación, identificado como ESP32-B, fue diseñado para recibir mensajes provenientes de múltiples dispositivos (A, C, D y E) mediante el protocolo ESP-NOW, los cuales se visualizan por medio de la pantalla OLED. Esta versión si permite una comunicación de los Nodos externos (uno a uno) con la parte central, y permite comunicar al Nodo Central con cualquiera de las ESP32 externas. Esto de manera local mediante WIFI del protocolo ESP-NOW.

El siguiente código corresponde a cualquiera de las 4 ESP32 restantes (A, C, D y E), ya que la estructura es similar para estas, debido a que son los nodos que se dirigen a la ESP-B (nodo central). Y los cambios más notables se ven en cuanto a la comunicación entre las ESP32, el cual se realiza de manera local, utilizando el protocolo ESP-NOW, no hay conexión via Wi-Fi con algún servidor. Las demás implementaciones se conservan de la misma manera (Enviar, Dormir y Sensor).

4.3 Documentación del código “ProtNowABCDEv4_ESP32-ACDE” y “ProtNowABCDEv4_ESP32-ACDE_Telegram”.

Este código está diseñado para los dispositivos ESP32 periféricos (A, C, D, E) que se comunican con el nodo central (ESP32-B) mediante el protocolo ESP-NOW, no gestiona múltiples dispositivos, ni Telegram, sino que se enfoca en enviar datos locales. Además, incorpora lógica avanzada para el sensor MPU6050 (calibración, detección de orientación y movimiento), lo que lo hace ideal para aplicaciones IoT distribuidas con monitoreo de movimiento.

Notas importantes

1. **Personalización por dispositivo:**
 - o Donde aparece "ESP32_X" o prefijos "X:", debe reemplazarse con la letra correspondiente al dispositivo (A, C, D, E)
 - o La MAC del dispositivo B (MAC_B) debe coincidir con la del dispositivo central en la red
 - o Los mensajes (MESSAGES) pueden personalizarse para cada dispositivo
2. **Configuración física:**
 - o Los pines de los botones (12, 13, 14) pueden necesitar ajuste según el hardware específico
 - o La configuración I2C (pines 21, 22) debe coincidir con la conexión física
3. **Funcionamiento:**
 - o El sistema tiene 3 estados principales: normal, menú y sensor
 - o En modo normal se pueden enviar mensajes predefinidos
 - o En modo sensor se envían datos del MPU6050 cuando se detecta movimiento (Cuenta con auto calibración que detecta la posición del sensor).
 - o El menú permite cambiar entre modos o entrar en deep sleep

Apartados del código: ProtNowABCDEv4_ESP32-ACDE (véase para mayor referencia el apartado de Anexos, de la página 58-63 a la 63-69, donde está este código en formato de colores y comentado).

Librerías importadas y sus propósitos

- Propósito: Importar las bibliotecas necesarias para el funcionamiento del sistema.
- Detalle:
 - o machine: Control de hardware (GPIO, I2C, deep sleep, RTC).
 - o network y espnow: Configuración de red WiFi y comunicación peer-to-peer.
 - o ssd1306: Manejo de pantallas OLED.

- o mpu6050: Lectura de datos del sensor MPU6050 (acelerómetro/giroscopio).
- o Funciones auxiliares: time, math, ubinascii, esp32.

Configuración OLED

- Propósito: Inicializar la pantalla OLED para mostrar información.
- Detalle:
 - o Configura el bus I2C (pines 21 y 22) y el objeto oled para controlar la pantalla.

Configuración MPU6050

- Propósito: Preparar el sensor MPU6050 para medir movimiento y orientación.
- Detalle:
 - o Inicializa el sensor y define umbrales para detección de movimiento (MOVE_THRESHOLD) y calibración (CALIBRATION_SAMPLES).

Variables para auto-detección de orientación

- Propósito: Determinar la orientación física del dispositivo (ejes X, Y, Z).
- Detalle:
 - o orientation: Almacena la orientación detectada (ej. "HORIZONTAL_Z_UP").
 - o calibration_done: Bandera que indica si la calibración inicial fue completada.

Configuración WIFI (Esta configuración se usa con la versión con conexión a Telegram)

- Propósito: Conectar el ESP32 correspondiente, a la red wifi para poder funcionar correctamente con enviando datos que serán recibidos por el ESP32 central y enviara a su vez al servidor de Telegram.
- Detalles: Reemplazar con los datos de la red a la que se conectaran (SSID y PASSWORD).

Configuración de botones

- Propósito: Implementar una clase para manejar botones con debounce.
- Detalle:
 - o La clase Button evita rebotes (debounce) y detecta pulsaciones confiables.

Inicializa botones (pines pueden variar según ESP32 específico)

- Propósito: Asignar pines GPIO a botones específicos.
- Detalle:
 - o btn_send (pin 12), btn_select (pin 14), btn_menu (pin 13).
 - o *Nota:* Los pines deben coincidir con el hardware de cada ESP32 (A, C, D, E).

Configuración ESP-NOW

- Propósito: Establecer comunicación inalámbrica entre dispositivos ESP32.

- **Conexión a red Wifi (Esta configuración se usa con la versión con conexión a Telegram):** llama las variables SSID y PASSWORD para conectar a la red.
- Detalle:
 - Inicializa la interfaz WiFi en modo estación (STA_IF) y activa ESP-NOW.

Configuración de peer (MAC debe ser actualizada según dispositivo B específico).

- Propósito: Definir el dispositivo receptor (ESP32_B) en la red.
- Detalle:
 - MAC_B: Dirección física del dispositivo central (debe ser actualizada).
 - e.add_peer(): Añade el peer para comunicación.

Mensajes disponibles

- Propósito: Lista de mensajes predefinidos que pueden enviarse.
- Detalle:
 - MESSAGES: Array con opciones personalizables (ej. "Hola", "Mensaje recibido").
 - current_msg: Índice del mensaje seleccionado.

Estados del sistema

- Propósito: Controlar el flujo del programa (normal, menú, sensor).
- Detalle:
 - STATE_NORMAL: Envío de mensajes.
 - STATE_MENU: Navegación de opciones.
 - STATE_SENSOR: Envío de datos del MPU6050.

Configuración wakeup para deep sleep (pin puede variar según ESP32)

- Propósito: Permitir que el dispositivo despierte desde modo bajo consumo.
- Detalle:
 - Configura el pin 12 (btn_send) como fuente de wakeup.
 - *Nota:* El pin debe coincidir con el hardware usado.

Mensaje inicial (personalizable para cada ESP32: A, C, D, E)

- Propósito: Mostrar un mensaje de inicio en la pantalla OLED.
- Detalle:
 - Ejemplo: "ESP32_X Lista" (reemplazar "X" por A, C, D, E).
 - Muestra la MAC del dispositivo y las instrucciones básicas.

Bucle principal

- Propósito: Gestionar la lógica del sistema en tiempo real.
- Detalle:
 - Monitoriza pulsaciones de botones para cambiar entre estados.
 - En STATE_NORMAL: Envía mensajes.
 - En STATE_SENSOR: Envía datos del MPU6050 si hay movimiento.

- o En STATE_MENU: Permite seleccionar opciones (sleep, sensor, etc.).
- o Recibe mensajes entrantes y los muestra en la OLED.

Este es el código para cualquiera de los nodos de la red de comunicación (A, C, D, E), el cual es modular y versátil, para poder establecer un punto nodal, que envíe mensajes y datos recabados por medio del sensor MPU6050 (giroscopio y acelerómetro). Notar que este código también se utiliza para la versión del proyecto que se conecta a un servidor en Telegram, en donde solo se hacen 2 agregados al código, para poder funcionar correctamente en dicha implementación de versión.

4.4 Documentación del código MPU6050

Los drivers necesarios para el funcionamiento correcto de los periféricos fueron 2:

➤ Sensor MPU6050

(El código en su formato MicroPython está en la página 69-71)

Este código es una librería completa para interactuar con el sensor MPU6050 (acelerómetro + giroscopio) en dispositivos ESP32 usando MicroPython. Su función principal es leer y procesar datos de movimiento (aceleración, rotación y temperatura) con configuraciones personalizables. Aquí su visión panorámica:

4.5 Documentación del código ssd1306

➤ Pantalla OLED

(El código en su formato MicroPython está en la página 71-73)

Esta librería permite controlar pantallas OLED monocromáticas de 128x64 píxeles mediante comunicación I2C en microcontroladores como el ESP32. Su función principal es manejar la configuración, escritura y visualización de gráficos o texto en la pantalla, actuando como una capa de abstracción entre el hardware y el software.

4.6 Conexión con el bot de Telegram

Creación del Bot

Para automatizar los procesos de comunicación usuario-sistema del proyecto, usó un agente automático que proporciona la aplicación de Telegram (bot). Estos agentes permiten establecer ciertos comportamientos o respuestas dados ciertos parámetros resultantes de la interacción de los usuarios, ya sea mandar mensajes automáticamente, mandar archivos con comandos o incluso la realización de pasarelas de pago incluidas en un chat bot de Telegram.

- 1) En primer lugar, se debe tener una cuenta en la aplicación de Telegram, para posteriormente buscar dentro de la aplicación el bot: @BotFather, el cual permitirá crear un sinfín de bots, configurar y personalizarlos. Una vez que encuentran, se debe iniciar presionando el botón de “iniciar”

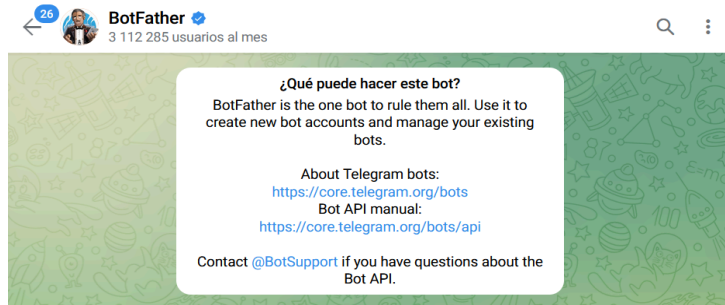


Figura 6. Primera interacción con BotFather.

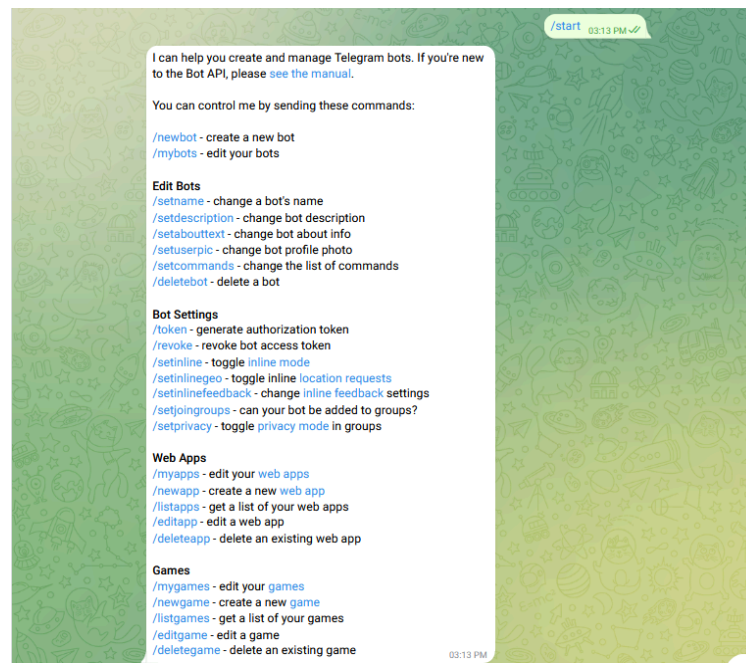


Figura 7. Iniciación de BotFather.

- 2) Posteriormente, se escribe el comando /newbot. Se podrá visualizar que se muestran comandos relacionados a lo que se escribe. Se deberán otorgar nombres al bot. En primer lugar, se solicita un nombre general, posteriormente, se solicitará un nombre de usuario con el que se podrá buscar al bot en el buscador integrado de telegram, este nombre debe terminar de la siguiente forma: “_bot”. Con ello, @BotFather generará el TOKEN

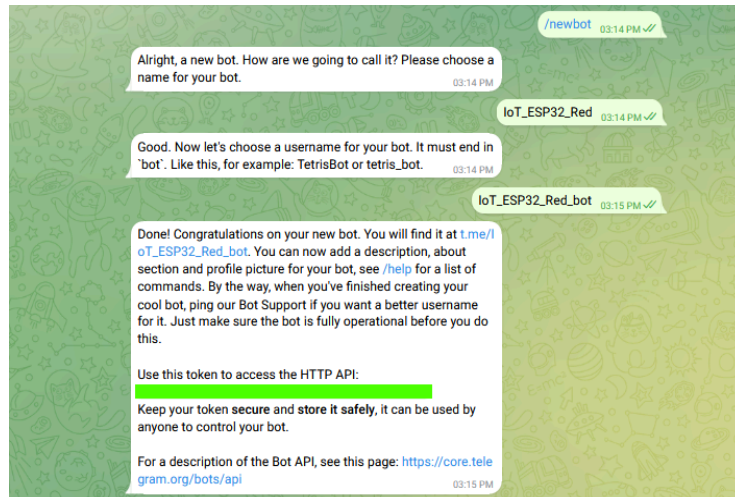


Figura 8. Creación del bot y grupo.

Nota: No compartir la HTTP API, es de uso personal y solo los implicados en el proyecto deben saberla, por temas de seguridad básica (De lo contrario, cualquier persona podría usar el bot).

- Consecuentemente, el bot estará listo para ser personalizado, por lo tanto, se puede buscar dentro de Telegram mediante el formato de nombre de usuario, el cual luce de la siguiente manera: "@botName_bot", en este caso: "@IoT_ESP32 Proyecto Terminal 25i".

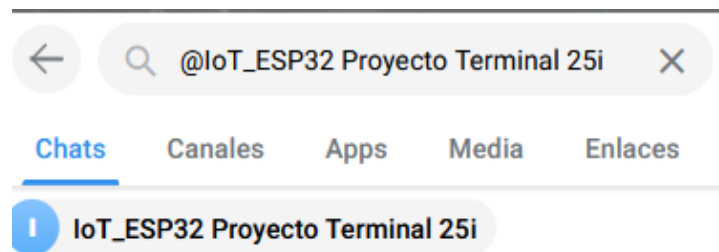


Figura 9. Búsqueda del grupo creado en el buscador de Telegram.

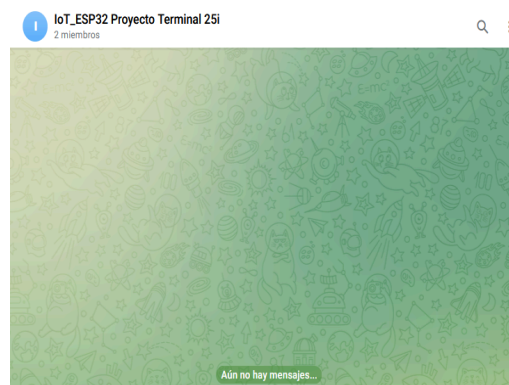
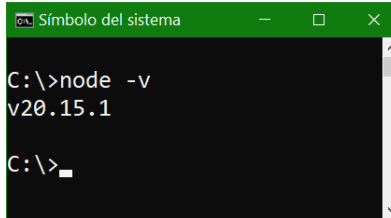


Figura 10. Grupo en Telegram implementado.

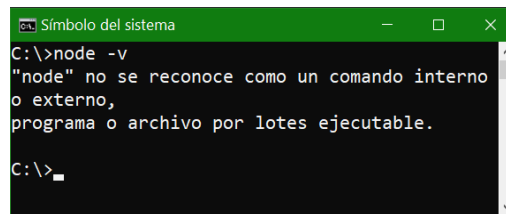
4.6.1 API para bots de Telegram en NodeJs



```
C:\>node -v
v20.15.1
C:\>
```

Figura 11. Comprobación de esta de NodeJs

Si NodeJS no está instalado se mostrará el siguiente mensaje



```
C:\>node -v
"node" no se reconoce como un comando interno
o externo,
programa o archivo por lotes ejecutable.
C:\>
```

Figura 12. Visualización si no está instalado NodeJs

En este caso, se recomienda instalar la última versión de NodeJS desde su página oficial <https://nodejs.org/en>

INICIALIZACIÓN DE UN PROYECTO

- 1) Una vez instalado NodeJS, se tendrá que iniciar un nuevo proyecto. En primer lugar, se debe crear la carpeta donde se guardarán los archivos, para posteriormente abrir esta carpeta en Visual Studio Code, y con su terminal, ejecutar el siguiente comando para iniciar el proyecto:

```
\telegram-nodejs-test1> npm init -y
```

- Al instalar librerías en el proyecto, se creará automáticamente la carpeta "node_modules" (la cual contiene todos los ficheros necesarios para ejecutar el proyecto, no interferir en esta carpeta) y se creará el archivo package-lock.json

Instalación de librerías

- 1) La librería con la que se trabaja para obtener los metadatos de conversaciones es “node-telegram-bot-api”, se puede instalar esta librería con el siguiente comando desde la terminal de Visual Studio Code, dentro de la carpeta del proyecto:

```
n\telegram-nodejs-test1> npm i node-telegram-bot-api
```

- 2) Para mayor seguridad y evitar colocar el token del bot directamente en el código, se instalará la librería: “dotenv”, la cual permitirá gestionar rápidamente ficheros .env, en donde se almacenan datos sensibles:

```
n\telegram-nodejs-test1> npm i dotenv
```

- 3) Para permitir que el programa se ejecute automáticamente o al guardar cambios, se hará uso de la librería “nodemon” :

```
n\telegram-nodejs-test1> npm i nodemon
```

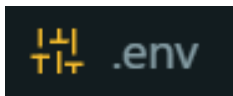
- Se recomienda agregar un script de ejecución en el fichero package.json para automatizar la ejecución de nodemon, y, por ende, del programa:

```
"scripts": {  
  "start": "nodemon ./index.js"  
},
```

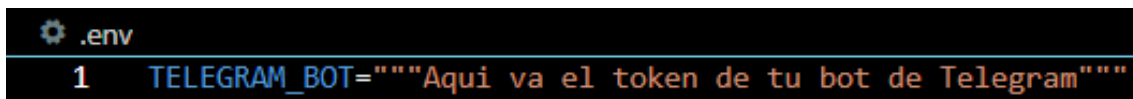
- 4) Posteriormente, se deberán crear los siguientes ficheros dentro del proyecto:



Fichero JavaScript index.js, es el punto de entrada de la aplicación NodeJS.



Fichero .env, almacena las variables de entorno en un archivo, aquí es donde se almacena el token del bot.



Uso de la librería

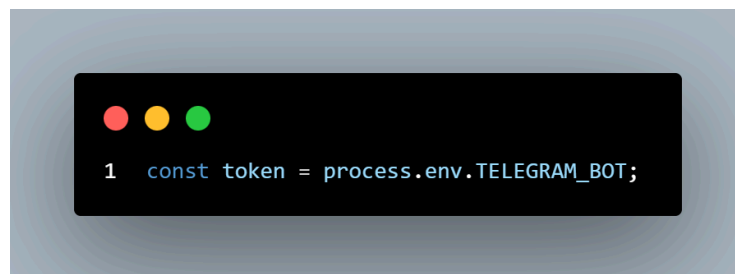
- 1) Dentro del fichero "index.js", se importa la librería que permite manipular el bot de telegram. El método `require()` forma parte de la importación de módulos en el formato CommonJS, este mismo módulo es guardado en la variable "TelegramBot". Posteriormente, importa la librería `dotenv`, y mediante el método `config()` carga todo archivo `.env` del proyecto



```
1 const TelegramBot = require('node-telegram-bot-api');
2 require('dotenv').config();
```

Figura 13. Primera parte del archivo index.js

- 2) Posteriormente, se crea una variable constante "token", aquí es donde se almacenará el token del bot directamente recuperado del fichero `.env`; una vez cargado el fichero mediante los métodos `require("dotenv").config()`, NodeJS tiene ya conocimiento de ese token, por lo que lo guarda en el objeto global "process", específicamente en la sección ".env" con la propiedad "TELEGRAM_BOT1".



```
1 const token = process.env.TELEGRAM_BOT1;
```

Figura 14. Variable que almacena el Token del servidor.

- 3) Ahora, se tiene que crear un nuevo objeto a partir de la clase "TelegramBot", los parámetros que recibe este objeto son: el token, así como un objeto de configuraciones adicionales, en este caso solamente se especifica el parámetro "polling" (para ver la lista completa de parámetros visite:
github.com/yagop/node-telegram-botapi/blob/master/doc/api.md#new_TelegramBot_new).

```
1 // CREAR UN NUEVO OBJETO TelegramBot
2 const bot = new TelegramBot(token, {polling: true});
```

Figura 15. Objetos que reciben al token.

- 4) Una vez instalado el objeto TelegramBot y configurado las variables de entorno, ya se puede trabajar con el bot de Telegram. Esta librería ofrece una amplia gama de funciones con las que se pueden realizar acciones desde responder automáticamente dados ciertos mensajes y eventos propios de Telegram (por ejemplo, el evento “onMessage” o “mensaje recibido”), copiar automáticamente mensajes, mandar audios, archivos, etc.

En este caso, se usará el método “onText()” para poder responder al usuario dado un comando enviado.

```
1 bot.onText('/start', (msg) => {
2   const chatId = msg.chat.id;
3   console.log('Received /start command from chat ID: ${chatId}');
4   bot.sendMessage(chatId, "Welcome to the bot! Type /help to see available commands.");
5 });
```

Figura 16. Función que recibe la Id del chat bot.

- Se visualizará de la siguiente manera desde la perspectiva del usuario que usa el bot y escribe el comando “/start”.

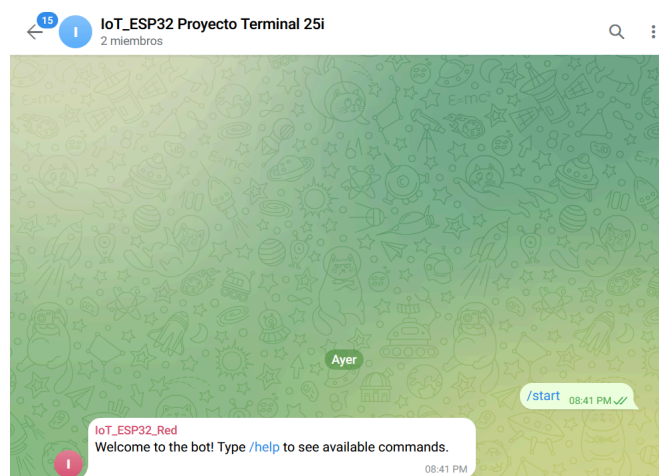
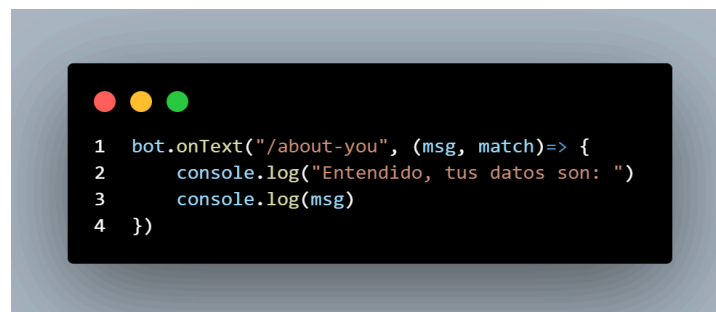


Figura 17. Visualización del grupo con el bot en funcionamiento.

Metadatos de conversaciones

Para obtener el id de conversación, se aprovechó las bondades de la librería “node-telegram-bot-api”, mediante la función `bot.onText()`, esta función permite que el bot “escuche” o esté al pendiente de mensajes especificados dentro de esta función, a continuación, se explica el procedimiento:

En primera instancia, se usa el método “`onText()`”, el primer parámetro que recibe es una cadena de texto que permite identificar el texto que desencadenará una función; por otro lado, el segundo parámetro es un callback o función, a su vez, este mismo trabajará con 2 argumentos (los metadatos completos del mensaje recibido y el texto adicional a los comandos establecidos, este último no se usa).

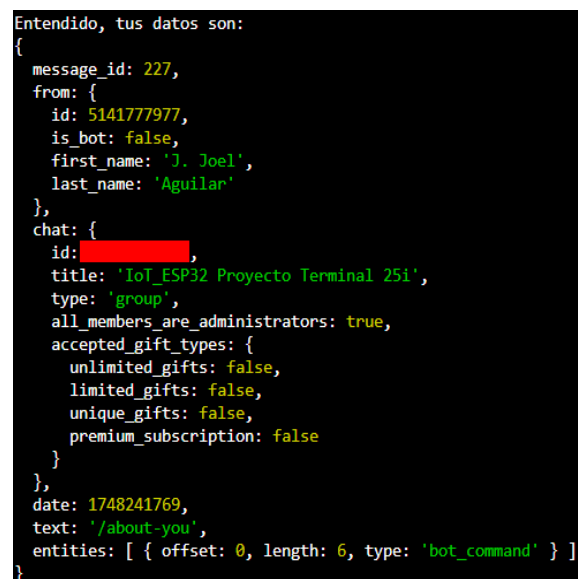
A screenshot of a code editor with a dark background and light-colored text. The code is written in JavaScript and uses the Telegram Bot API library. It shows the `bot.onText()` method being used with a command `"/about-you"` and a callback function. The callback function logs the message text to the console.

```
1 bot.onText("/about-you", (msg, match)=> {
2   console.log("Entendido, tus datos son: ")
3   console.log(msg)
4 })
```

Figura 18. Uso de la librería `onText`.

Posteriormente y primero, se muestra por consola una cadena de texto para posteriormente mostrar el objeto “`msg`”.

Se mostrará por consola un mensaje de este tipo. Donde el `chat_id` será aquel que esté en el apartado que se ve en color rojo (este es un id propio que no debe transferirse por cuestiones de seguridad del proyecto).

A screenshot of a console log showing the JSON object returned by the Telegram Bot API when a command is received. The object contains details about the message, the sender, and the chat. The `chat_id` field is highlighted in red in the original image.

```
Entendido, tus datos son:
{
  message_id: 227,
  from: {
    id: 5141777977,
    is_bot: false,
    first_name: 'J. Joel',
    last_name: 'Aguilar'
  },
  chat: {
    id: [REDACTED],
    title: 'IoT_ESP32 Proyecto Terminal 25i',
    type: 'group',
    all_members_are_administrators: true,
    accepted_gift_types: {
      unlimited_gifts: false,
      limited_gifts: false,
      unique_gifts: false,
      premium_subscription: false
    }
  },
  date: 1748241769,
  text: '/about-you',
  entities: [ { offset: 0, length: 6, type: 'bot_command' } ]
}
```

Figura 19. Obtención de los metadatos del bot de Telegram.

5. Discusión de resultados

5.1 La mejora del proyecto con la implementación de algún otro sensor

De acuerdo con los alcances de este proyecto, sólo se ha enfocado en el uso del sensor MPU6050, pero puede adaptarse a algún otro sensor como puede ser el de Temperatura y Humedad (DHT11), o el sensor de gas y humo (MQ-2).

5.2 Operación del sistema en su primera versión (comunicación local con ESP-NOW sin uso de alguna red wifi externa)

Esta primera version fue realizada con los codigos:

- ProtNowABCDEv4_ESP32-B
- ProtNowABCDEv4_ESP32-ACDE

Donde la interacción se realiza de manera local, sin necesidad de Wi-Fi. Los nodos exteriores se comunican enviando mensajes y datos inerciales al centro. Y el ESP32 central funciona como receptor de todos los datos, y a su vez tiene la función de mandar mensaje a cualquiera de los nodos exteriores (A, C, D y E).

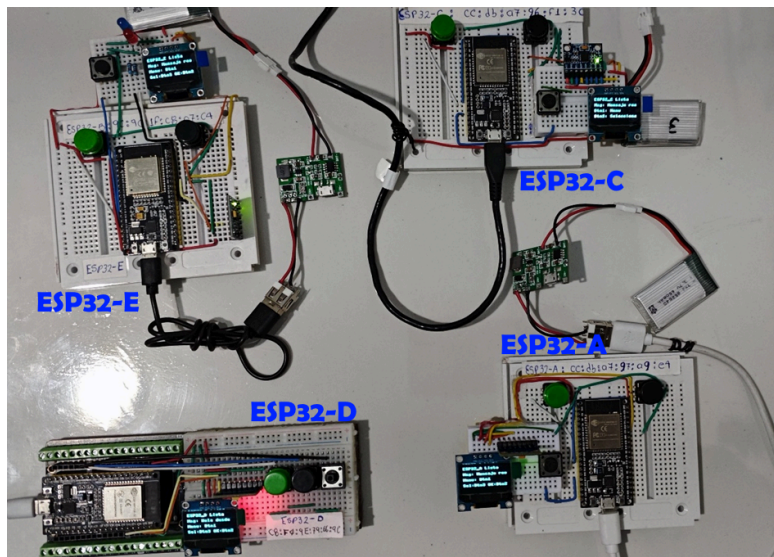


Figura 20. Conjunto de ESP32 (A,C,D,E) en funcionamiento local (protocolo ESP-NOW).

Notar que la ESP-C es en la que se implementa la posición del sensor diferente, estando esta horizontal con el eje z apuntando hacia arriba (su parte positiva).

Para los nodos (A,C,D,E) el menú de opciones a realizar es el mismo, cabe señalar.

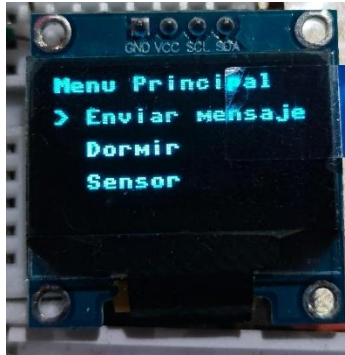


Figura 21. Menú con las opciones disponibles para ESP32-ACDE

Una vez en funcionamiento, la red de ESP32 se completa con el nodo central, empezando a recibir mensajes y datos inerciales.

En la siguiente imagen se puede observar el resultado de recibir datos de la ESP-A, por lo cual, según la implementación y arreglo de los leds indicadores, se enciende el de color azul.

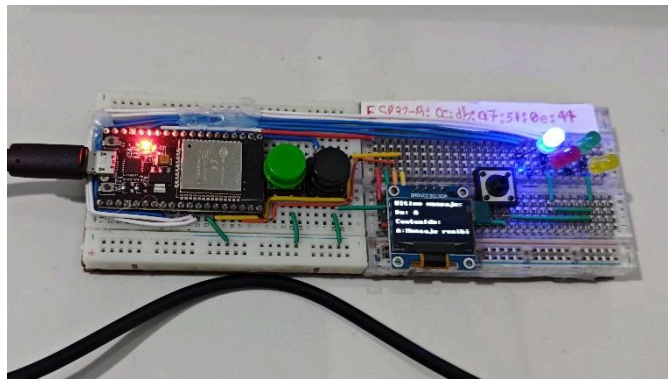


Figura 22. ESP32-B recibe datos de ESP32-A

Ahora se observa la llegada de los datos de la ESP32-C, indicado por el led rojo.

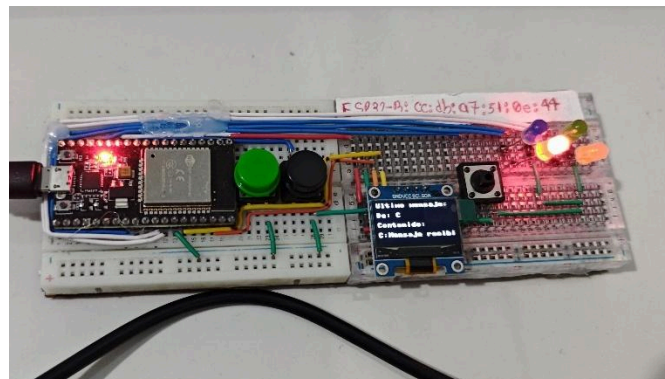


Figura 23. ESP32-B recibe datos de ESP32-C

El siguiente resultado muestra los datos provenientes de la ESP32-D, el cual se ve indicado por el led verde.

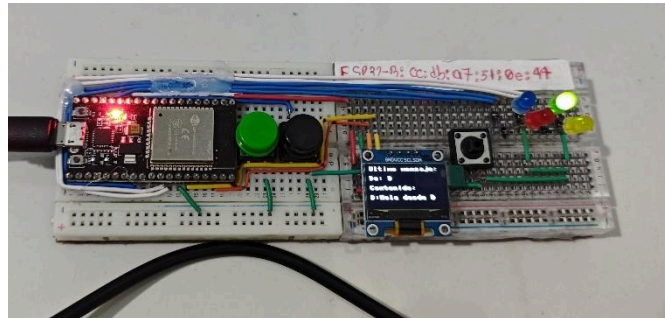


Figura 24. ESP32-B recibe datos de ESP32-D

Por último, se puede ver la llegada de los datos de la ESP32-E, indicado por el led amarillo.

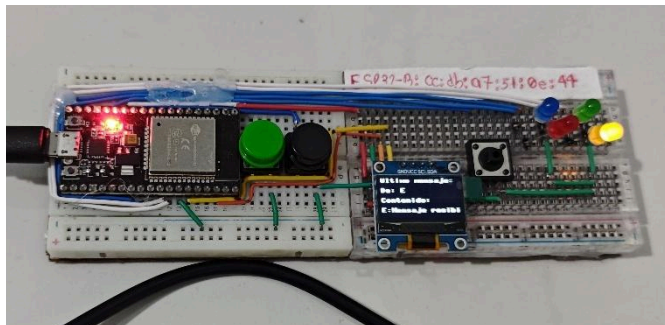


Figura 25. ESP32-B recibe datos de ESP32-E

Estos resultados son de manera local, ósea, la comunicación se realiza entre la red de 5 ESP32 (A,C,D,E) con la parte central (B), sin conexión a una red externa wifi, aprovechando totalmente una conexión peer to peer.

Por último, cabe mencionar que cada uno de los nodos ESP32 (A,B,C,D,E) cuenta con la función de entrar en modo deepsleep, el cual pone al sistema en modo ahorro de energía, dejando de realizar tareas y procesos hasta que se reactive con el GPIO designado (12 en este caso, o botón verde).

5.3 Operación de un nuevo sistema propuesto (comunicación con el servidor de Telegram).

El comportamiento de la ESP32-B aquí al recibir mensajes y datos inerciales, es el ya conocido, encendiendo un led según de donde vengan los datos.

Azul: A, Rojo: C, Verde: D y Amarillo: E. Por lo cual los resultados más destacables se observan en el grupo de Telegram designado (IoT_ESP32 Proyecto Terminal 25i).

La siguiente figura es una visualización de lo que manda ESP32-B al servidor de Telegram, reportando en ese momento que función está realizando.

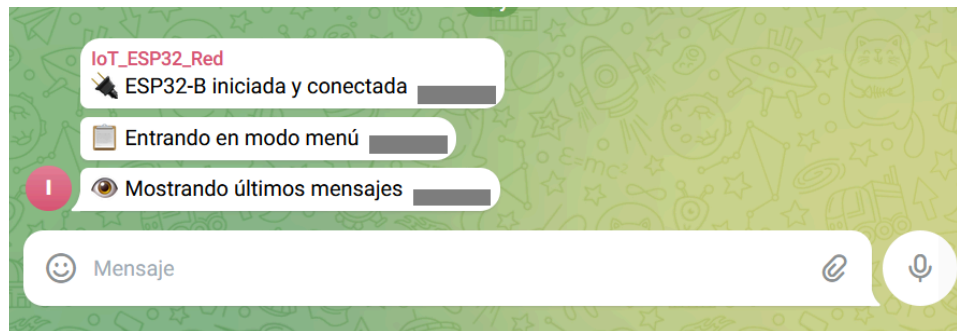


Figura 26. Visualización en Telegram del ingreso a menús disponibles.

El menú de opciones para ESP32-B es el mismo que para la primera implementación (local con solo protocolo ESP-NOW), con la diferencia de que aquí (solamente) la opción de enviar mensajes no está siendo implementada (las dos restantes sí).

La opción de llevar el sistema a deepsleep, si funciona correctamente, lo cual lleva al modo de ahorro de energía y deja al sistema de (B) sin funcionamientos, hasta que se reinicie el sistema con el GPIO asignado (12 en este caso también, o botón verde)



Figura 27. Menú con las opciones disponibles para ESP32-B

Una vez iniciada la comunicación con el servidor, y estando el modo ver mensajes, el sistema estará en espera de los datos provenientes de (A,C,D,E).

Para mayor ilustración en la siguiente figura se colocó un indicador de color que corresponde a una asignación visual que concuerda con el color de led que se activa según sea de cual ESP32 viene el mensaje/dato.

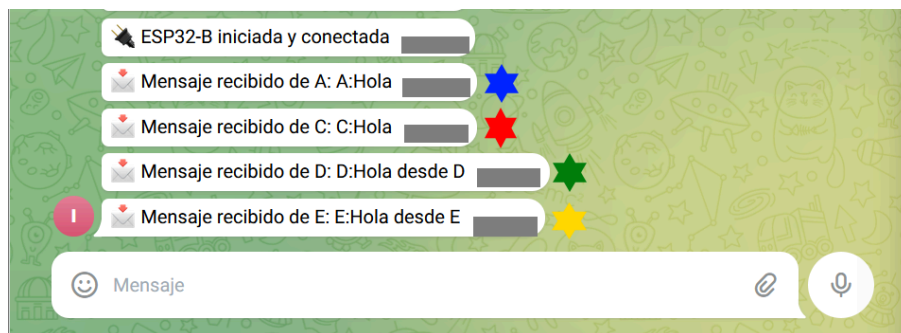


Figura 28. Mensajes recibidos reportados en el servidor de Telegram.

Al activar el sensor de los nodos externos (A,C,D,E), en ESP32-B se visualizarán los datos inerciales, como se ve en la siguiente figura.

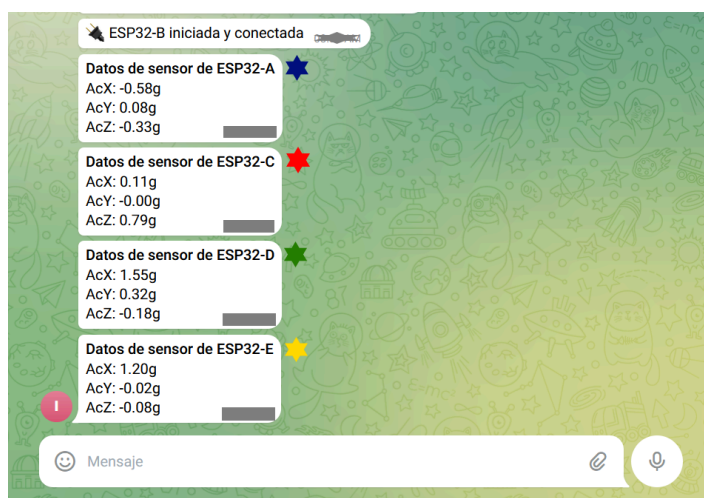


Figura 29. Datos inerciales reportados en el chat bot de Telegram.

Por último, la visualización del funcionamiento del modo deepsleep, también puede reportarse y enviarse al chat bot.

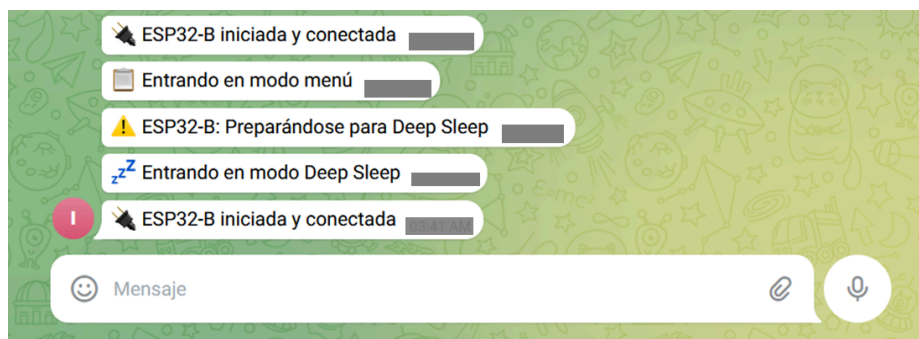


Figura 30. Modo deepsleep reportado al chat bot de Telegram.

6. Mejoras a futuro y Conclusiones

La implementación de una solución para el conflicto entre la conexión al servidor (Telegram) y la comunicación local entre B y los demás nodos (A,C , D, E).

Resulta que surgió un problema con la comunicación, donde se hay conflicto entre la conexión Wi-Fi al servidor de Telegram y el protocolo ESP-NOW para interconectar los nodos de la red (en específico el ESP32-B mandando mensajes hacia las otras de la red: A, C, D y E). Al momento de estar conectado con el servidor en Telegram, la ESP-B (la central), no puede realizar la función de enviar mensaje a las demás. Como complemento y mejora a esta parte del proyecto, quien lo retome podría buscar alguna implementación para que puedan subsistir estas 2 funciones, o crear algún switch para poder intercambiar entre estos 2 estados.

Las alterativas a implementar son 2, una es buscar el uso de canales entre los ESP32, para que no haya conflicto con la conexión a internet. Y la segunda manera es ver la forma de que se intercale la comunicación peer to peer, con la conexión a una red Wifi.

Aclaración sobre el sensor y su posición en que se instala (a mi parecer es mejor en forma vertical).

El sensor MPU6050 se implementó de tal manera que al iniciarse la utilización de este por parte de alguna de las ESP32 de los nodos exteriores (A, C, D y E), empezará con una auto calibración, para detectar en qué posición se encuentra en ese momento. Bajo esta configuración aún de esta manera, se observó que una colocación vertical (eje x hacia arriba), responde mejor y no hay tantas lecturas demasiado sensibles. Y si, por el contrario, se colocaba de manera horizontal. Este presentaba lecturas sensibles con movimientos poco fuertes. Esto es buscar mas por el lado de la calibración y ajustes en los datos inerciales.

Versatilidad técnica de sistema IoT combinando ESP32 y MPU6050

Se pudo demostrar la efectividad del uso combinado de microcontroladores ESP32 y sensores MPU6050 para crear las redes de monitoreo inercial, lo que valida el que tecnologías de bajo costo puedan implementarse en proyectos IoT , y con ello obtener resultados adecuados y precisos. La integración de protocolos ESP-NOW e I2C permitió una comunicación estable entre nodos, aunque se identificó que el conflicto entre Wi-Fi y ESP-NOW limita el desempeño cuando se

quiere hacer una conexión simultánea a internet (en la conexión con el chat bot de Telegram).

Alcances y escalabilidad

El proyecto sirve como base para aplicaciones académicas en electrónica embebida, permitiendo a quien lo retome, explorar desde conceptos básicos (GPIO, PWM) hasta sistemas distribuidos complejos. Su arquitectura modular admite la incorporación de nuevos sensores (como DTH11 para temperatura /humedad o MQ-2 para la calidad del aire), lo que amplía su potencial en áreas como agricultura de precisión, domótica o monitoreo estructural.

Limitaciones y oportunidades de mejora.

La calibración del sensor MPU6050 requiere ajustes específicos según su orientación física, lo que de alguna forma complejiza un poco más la utilización de este sensor. Y eso que no se profundizó más en la calibración de sensibilidad o en la detección de algún movimiento en específico (ondulaciones o inclinaciones más específicas).

El consumo energético en modo activo limita la autonomía, por lo cual aun y en la aplicación de la utilización de un sistema de alimentación por batería, surgió la necesidad de optimizar mediante protocolo de hibernación (deepsleep). Como posible mejora y adaptación, se podría buscar el aplicar esto mediante ciclos de tiempo, que activen durante ciertos periodos el funcionamiento de la red, o también mediante la interrupción recibida por el sensor MPU6050 (alguna sacudida que active el sistema y empiece a registrar).

El actual sistema no almacena datos localmente si llegase a fallar la conexión a Telegram, lo que podría significar una pérdida de información. Una solución viable sería incorporar un módulo EEPROM o tarjeta SD para guardar registros temporalmente y sincronizarlos cuando se reestablezca la comunicación.

Mejoras en la interfaz de usuario, para que la navegación mediante botones físicos y pantalla OLED, que no permite visualizar históricos o gráficos de los datos del sensor, podría mejorarse con la implementación de una interfaz web local (usando el modo AP del ESP32), o botones táctiles permitiendo interfaces más intuitivas.

Anexos

Código: ProtNowABCDEv4.1 ESP32-B

"""

Alumno: José Joel Aguilar de la Cruz

Carrera: Ing. Electrónica

Matricula: 2173049871

Programa: ProtNowABCDEv4.1_ESP32-B

Descripcion: El código implementa un sistema de comunicación entre varios dispositivos ESP32 utilizando el protocolo ESP-NOW, para poder intercambiar información recabada con el sensor MPU6050 y enviarla a un servidor en línea, en este caso el utilizado es Telegram.

Proyecto terminal:

Proyecto Terminal IoT ESP32:

Red de comunicación y monitoreo mediante sensores (MPU6050)

UAMI, CDMX mayo 2025

"""

"""Librerías Utilizadas"""

```
from machine import Pin, SoftI2C, deepsleep, RTC # Control de hardware: GPIO, I2C, deepsleep y RTC
import network # Manejo de conexiones WiFi
import espnw # Protocolo ESP-NOW para comunicación peer-to-peer
import ssd1306 # Control de pantalla OLED
import time # Funciones de tiempo y delays
import ubinascii # Conversión de datos binarios/hexadecimales
import machine # Funciones básicas del microcontrolador
import esp32 # Funciones específicas del ESP32
import urequests # Peticiones HTTP (para Telegram)
import json # Manejo de datos JSON
```

"""Configuración OLED"""

```
# Configuración del bus I2C y pantalla OLED (128x64 píxeles)
i2c = SoftI2C(scl=Pin(22), sda=Pin(21)) # Pines 22 (SCL) y 21 (SDA)
oled = ssd1306.SSD1306_I2C(128, 64, i2c) # Inicialización de la pantalla
```

"""Configuración de LEDs"""

```
# LEDs para indicar mensajes recibidos de cada dispositivo
led_a = Pin(16, Pin.OUT) # LED para mensajes de A
led_c = Pin(17, Pin.OUT) # LED para mensajes de C
led_d = Pin(18, Pin.OUT) # LED para mensajes de D
led_e = Pin(19, Pin.OUT) # LED para mensajes de E
```

"""Configuración WiFi y Telegram"""

```
# Credenciales WiFi (deben ser reemplazadas)
WIFI_SSID = "tu_ssid"
WIFI_PASSWORD = "tu_password"
```

```

# Configuración del bot de Telegram
TELEGRAM_TOKEN = "tu_token_bot" # Token del bot proporcionado por BotFather
TELEGRAM_CHAT_ID = "tu_chat_id" # ID del chat donde se enviarán los mensajes

"""Clase para manejo de botones"""
class Button:
    def __init__(self, pin):
        self.pin = Pin(pin, Pin.IN, Pin.PULL_UP) # Configura pin como entrada con
        pull-up
        self.last_press = 0 # Tiempo del último press detectado
        self.debounce = 350 # Tiempo de debounce en ms

    def pressed(self):
        # Detecta si el botón está presionado (considerando debounce)
        if self.pin.value() == 0 and (time.ticks_ms() - self.last_press) > self.debounce:
            self.last_press = time.ticks_ms()
            return True
        return False

# Instancias de botones
btn_menu = Button(13) # Botón para menú/volver
btn_ok = Button(12) # Botón para confirmar acciones
btn_select = Button(14) # Botón para seleccionar opciones

""" Configuración ESP-NOW """
sta = network.WLAN(network.STA_IF) # Interfaz WiFi en modo estación
sta.active(True) # Activa la interfaz WiFi
e = espnow.ESPNow() # Instancia de ESP-NOW
e.active(True) # Activa ESP-NOW

# Direcciones MAC de los dispositivos pares (A, C, D, E)
MAC_A = ubinascii.unhexlify('CCDBA797A9E4'.replace(':', ''))
MAC_C = ubinascii.unhexlify('CCDBA796F13C'.replace(':', ''))
MAC_D = ubinascii.unhexlify('C8F09E79669C'.replace(':', ''))
MAC_E = ubinascii.unhexlify('9C9C1FC807C4'.replace(':', ''))

# Agrega los dispositivos como pares
e.add_peer(MAC_A)
e.add_peer(MAC_C)
e.add_peer(MAC_D)
e.add_peer(MAC_E)

""" Estados del sistema """
# Estados posibles de la máquina de estados
STATE_MONITOR = 0 # Estado inicial: monitoreo de mensajes
STATE_MENU = 1 # Menú principal
STATE_SEND_TARGET = 2 # Selección de destino para enviar mensaje
STATE_SEND_MSG = 3 # Selección de mensaje a enviar
STATE_SLEEP_CONFIRM = 4 # Confirmación para entrar en deep sleep
STATE_SENSOR_DISPLAY = 5 # Visualización de datos de sensores

```

```

current_state = STATE_MONITOR # Estado inicial

""" Variables de control"""
current_option = 0 # Opción actual seleccionada en menús
last_messages = [] # Historial de últimos mensajes recibidos
selected_target = 0 # Dispositivo seleccionado para enviar mensaje
sensor_data = None # Datos de sensor recibidos
wifi_connected = False # Estado de conexión WiFi

""" Mensajes Predefinidos"""
MESSAGES = [
    "Te copio, mensaje recibido",
    "Enviame un mensaje",
    "Ya no envíes mensaje",
    "HOLI xd"
]

"""Funciones de conexión WiFi y Telegram"""
def connect_wifi():
    """Intenta conectar a WiFi y actualiza el estado global"""
    global wifi_connected
    if not sta.isconnected():
        print('Conectando a WiFi...')
        sta.active(True)
        sta.connect(WIFI_SSID, WIFI_PASSWORD)

        start_time = time.time()
        while not sta.isconnected() and (time.time() - start_time) < 10:
            time.sleep(0.5)

        if sta.isconnected():
            print('WiFi conectado:', sta.ifconfig())
            wifi_connected = True
            return True
        else:
            print('Error conectando WiFi')
            wifi_connected = False
            return False
    wifi_connected = True
    return True

def send_telegram_message(message):
    """Envía un mensaje al chat de Telegram configurado"""
    global wifi_connected

    if not wifi_connected and not connect_wifi():
        print("No se pudo conectar a WiFi")
        return False

    url = "https://api.telegram.org/bot{}/sendMessage".format(TELEGRAM_TOKEN)
    headers = {'Content-Type': 'application/json'}

```



```

data = {
    "chat_id": TELEGRAM_CHAT_ID,
    "text": message,
    "parse_mode": "Markdown"
}

try:
    response = urequests.post(url, headers=headers, json=data)
    print("Respuesta Telegram:", response.text)
    response.close()
    return True
except Exception as e:
    print("Error enviando a Telegram:", e)
    return False
finally:
    # Desconecta WiFi si no estamos en modo sensor o monitor
    if current_state not in [STATE_SENSOR_DISPLAY, STATE_MONITOR]:
        sta.active(False)
        wifi_connected = False

"""Funciones de visualizacion"""
def blink_led(led):
    """Hace parpadear un LED brevemente"""
    led.value(1)
    time.sleep(0.2)
    led.value(0)

def clear_display():
    """Limpia la pantalla OLED"""
    oled.fill(0)
    oled.show()

def show_text(line1="", line2="", line3="", line4=""):
    """Muestra hasta 4 líneas de texto en la pantalla OLED"""
    clear_display()
    oled.text(line1, 0, 0)
    oled.text(line2, 0, 16)
    oled.text(line3, 0, 32)
    oled.text(line4, 0, 48)
    oled.show()

def show_monitor():
    """Muestra el estado de monitor con el último mensaje recibido"""
    if not last_messages:
        show_text("Esperando mensajes...", "Estado: Activo", "", "Menu: Btn1")
        return

    msg, origin, _ = last_messages[-1]
    show_text(
        "Ultimo mensaje:",
        f"De: {origin}",
        "Contenido:",

```

```

        msg[:16] + (msg[16:] and '...')
    )

    if len(last_messages) > 0:
        telegram_msg = f"📧 Mensaje recibido de {origin}: {msg}"
        send_telegram_message(telegram_msg)

def show_menu():
    """Muestra el menú principal con las opciones disponibles"""
    options = [
        "Menu Principal",
        "> Ver" if current_option == 0 else "  Ver",
        "> Enviar" if current_option == 1 else "  Enviar",
        "> Dormir" if current_option == 2 else "  Dormir",
        "> Sensor" if current_option == 3 else "  Sensor"
    ]
    show_text(options[0], options[1], options[2], options[3])

def show_send_target():
    """Muestra la selección de destino para enviar mensaje"""
    targets = ["A", "C", "D", "E"]
    title = "Enviar a:"

    if current_option < 3:
        options = [
            title,
            f"> {targets[0]}" if current_option == 0 else f"  {targets[0]}",
            f"> {targets[1]}" if current_option == 1 else f"  {targets[1]}",
            f"> {targets[2]}" if current_option == 2 else f"  {targets[2]}"
        ]
    else:
        options = [
            title,
            f"  {targets[1]}",
            f"  {targets[2]}",
            f"> {targets[3]}"
        ]

    show_text(options[0], options[1], options[2], options[3])

def show_send_message():
    """Muestra la selección de mensaje a enviar"""
    target_name = ["A", "C", "D", "E"][selected_target]
    msg_index = current_option % len(MESSAGES)
    show_text(
        f"Enviar a {target_name}:",
        f"> {MESSAGES[msg_index]}",
        "",
        "Ok: Btn2 Sel: Btn3"
    )

def show_sleep_confirm():

```

```

    """Muestra la confirmación para entrar en deep sleep"""
    show_text("Confirmar Deep Sleep", "Presione OK para", "entrar en modo", "sueno profundo")
    send_telegram_message("⚠ ESP32-B: Preparándose para Deep Sleep")

def show_sensor_data():
    """Muestra los datos de sensor recibidos"""
    if sensor_data:
        try:
            parts = sensor_data.decode().split(':')[2].split(',')
            origin = "A" if sensor_data.startswith(b'S:A') else \
                    "C" if sensor_data.startswith(b'S:C') else \
                    "D" if sensor_data.startswith(b'S:D') else \
                    "E"

            # Enciende el LED correspondiente al origen
            if origin == "A":
                blink_led(led_a)
            elif origin == "C":
                blink_led(led_c)
            elif origin == "D":
                blink_led(led_d)
            else:
                blink_led(led_e)

            if len(parts) == 6:
                show_text(
                    f"Datos de {origin}:",
                    f"AcX: {float(parts[0])/16384:.2f}g",
                    f"AcY: {float(parts[1])/16384:.2f}g",
                    f"AcZ: {float(parts[2])/16384:.2f}g"
                )

                telegram_msg = (
                    f"*Datos de sensor de ESP32-" + str(origin) + " *\n" +
                    f"AcX: {float(parts[0])/16384:.2f}g\n" +
                    f"AcY: {float(parts[1])/16384:.2f}g\n" +
                    f"AcZ: {float(parts[2])/16384:.2f}g"
                )
                send_telegram_message(telegram_msg)
            except Exception as e:
                print("Error procesando datos:", e)
                show_text("Error en datos", "del sensor", "", "")

        else:
            show_text("Esperando datos", "del sensor...", "", "Menu: Btn1")

    """Configuración de wakeup e inicialización"""
    # Configura el pin 12 como fuente de wakeup para deep sleep
    esp32.wake_on_ext0(pin=Pin(12, Pin.IN, Pin.PULL_UP), level=esp32.WAKEUP_ALL_LOW)

# Inicialización del sistema

```

```

#connect_wifi() #Conecta a WIFI, se puede comentar si no se requiere conexión inicial
send_telegram_message("🔌 ESP32-B iniciada y conectada")
show_text("Sistema iniciado", "Esperando mensajes...", "Estado: Activo", "Menu: Btn1")

"""Bucle principal"""
while True:
    try:
        # Recepción de mensajes ESP-NOW
        sender, msg = e.recv(500)
        if msg:
            if msg.startswith(b'S:'): # Datos de sensor
                sensor_data = msg
                if current_state == STATE_MONITOR or current_state ==
STATE_SENSOR_DISPLAY:
                    show_sensor_data()
            else: # Mensaje normal
                origin = "A" if sender == MAC_A else \
                    "C" if sender == MAC_C else \
                    "D" if sender == MAC_D else \
                    "E"

                last_messages.append((msg.decode(), origin, time.time()))

                # Enciende LED según origen
                if origin == "A":
                    blink_led(led_a)
                elif origin == "C":
                    blink_led(led_c)
                elif origin == "D":
                    blink_led(led_d)
                else:
                    blink_led(led_e)

                if len(last_messages) > 3:
                    last_messages.pop(0)
                if current_state == STATE_MONITOR:
                    show_monitor()

        # Manejo de botones y máquina de estados
        if btn_menu.pressed():
            # Lógica para manejar el botón menú en diferentes estados
            if current_state == STATE_MONITOR:
                current_state = STATE_MENU
                current_option = 0
                show_menu()
                send_telegram_message("📄 Entrando en modo menú")
            elif current_state == STATE_MENU:
                current_state = STATE_MONITOR
                show_monitor()
                send_telegram_message("👁 Volviendo al modo monitor")
            elif current_state == STATE_SLEEP_CONFIRM:
                current_state = STATE_MENU

```

```

        show_menu()
    elif current_state == STATE_SENSOR_DISPLAY:
        current_state = STATE_MONITOR
        show_monitor()
        time.sleep(0.3)

if current_state == STATE_MENU:
    # Lógica para navegar el menú principal
    if btn_select.pressed():
        current_option = (current_option + 1) % 4
        show_menu()
        time.sleep(0.3)

    if btn_ok.pressed():
        if current_option == 0: # Ver mensajes
            current_state = STATE_MONITOR
            show_monitor()
            send_telegram_message("👁️ Mostrando últimos mensajes")
        elif current_option == 1: # Enviar mensaje
            current_state = STATE_SEND_TARGET
            current_option = 0
            show_send_target()
            send_telegram_message("📤 Preparándose para enviar mensaje")
        elif current_option == 2: # Dormir
            current_state = STATE_SLEEP_CONFIRM
            show_sleep_confirm()
        else: # Ver datos de sensor
            current_state = STATE_SENSOR_DISPLAY
            show_sensor_data()
            send_telegram_message("📡 Mostrando datos de sensores")
        time.sleep(0.3)

elif current_state == STATE_SEND_TARGET:
    # Lógica para seleccionar destino de mensaje
    if btn_select.pressed():
        current_option = (current_option + 1) % 4
        show_send_target()
        time.sleep(0.3)

    if btn_ok.pressed():
        selected_target = current_option
        current_state = STATE_SEND_MSG
        current_option = 0
        show_send_message()
        time.sleep(0.3)

elif current_state == STATE_SEND_MSG:
    # Lógica para seleccionar y enviar mensaje
    if btn_select.pressed():
        current_option = (current_option + 1) % len(MESSAGES)
        show_send_message()
        time.sleep(0.3)

```

```

if btn_ok.pressed():
    target = [MAC_A, MAC_C, MAC_D, MAC_E][selected_target]
    target_name = ["A", "C", "D", "E"][selected_target]
    message = MESSAGES[current_option % len(MESSAGES)]
    e.send(target, f"B:{message}")

    telegram_msg = f"📩 Mensaje enviado a {target_name}: {message}"
    send_telegram_message(telegram_msg)

    show_text("Mensaje enviado!", f"A {target_name}:", message[:16], "")
    time.sleep(2)
    current_state = STATE_MONITOR
    show_monitor()
    time.sleep(0.3)

elif current_state == STATE_SLEEP_CONFIRM:
    # Lógica para confirmar deep sleep
    if btn_ok.pressed():
        show_text("Entrando en", "Deep Sleep...", "Despertar con", "Btn OK")
        send_telegram_message("💤 Entrando en modo Deep Sleep")
        time.sleep(2)
        deepsleep()
        time.sleep(0.3)

elif current_state == STATE_SENSOR_DISPLAY:
    # Estado de visualización de sensores (no requiere acciones adicionales)
    pass

except Exception as e:
    print("Error:", e)
    send_telegram_message(f"❌ Error en ESP32-B: {str(e)}")
    time.sleep(1)
    machine.reset()

```

Código: ProtNowABCDEv4_ESP32-B

"""

Alumno: José Joel Aguilar de la Cruz

Carrera: Ing. Electrónica

Matricula: 2173049871

Programa:ProtNowABCDEv4_ESP32-B

Descripcion: Programa para el manejo de mensajes entre dispositivos ESP32 mediante ESP-NOW. Que funciona como el nodo central de un sistema de monitoreo y control de sensores (MPU6050). La comunicación se realiza entre 4 dispositivos y este nodo central, de manera local (peer-to-peer) sin necesidad de un router o acceso a internet.

Proyecto terminal:

Proyecto Terminal IoT ESP32:
Red de comunicación y monitoreo mediante sensores (MPU6050)

UAMI, CDMX mayo 2025

"""

"""Librerias Utilizadas"""

```
from machine import Pin, SoftI2C, deepsleep, RTC # Control de hardware (GPIO, I2C, deep sleep, reloj)
import network # Manejo de conexiones WiFi
import espnow # Protocolo ESP-NOW para comunicación peer-to-peer
import ssd1306 # Control de pantalla OLED
import time # Funciones de tiempo y delays
import ubinascii # Conversión de datos binarios/hexadecimales
import machine # Funciones básicas del microcontrolador
import esp32 # Funciones específicas del ESP32 (ej. wakeup)
```

"""Configuración de la pantalla OLED"""

```
i2c = SoftI2C(scl=Pin(22), sda=Pin(21)) # Inicializa bus I2C en pines 22 (SCL) y 21 (SDA)
oled = ssd1306.SSD1306_I2C(128, 64, i2c) # Pantalla OLED de 128x64 píxeles
```

"""Configuracion de LEDs"""

```
led_a = Pin(16, Pin.OUT) # LED para mensajes del dispositivo A
led_c = Pin(17, Pin.OUT) # LED para mensajes del dispositivo C
led_d = Pin(18, Pin.OUT) # LED para mensajes del dispositivo D
led_e = Pin(19, Pin.OUT) # LED para mensajes del dispositivo E
```

"""Configuracion para manejo de botones """

```
class Button:
    def __init__(self, pin):
        self.pin = Pin(pin, Pin.IN, Pin.PULL_UP) # Configura pin como entrada con pull-up
```

```

        self.last_press = 0 # Tiempo del último press
        self.debounce = 350 # Tiempo de debounce en ms

    def pressed(self):
        # Detecta pulsaciones válidas (evita rebotes)
        if self.pin.value() == 0 and (time.ticks_ms() - self.last_press) > self.debounce:
            self.last_press = time.ticks_ms()
            return True
        return False

# Instancias de botones
btn_menu = Button(13) # Botón para entrar al menú/volver
btn_ok = Button(12)    # Botón para confirmar acciones
btn_select = Button(14) # Botón para navegar opciones

"""Configuración ESP-NOW"""
sta = network.WLAN(network.STA_IF) # Interfaz WiFi en modo estación
sta.active(True) # Activa la interfaz
e = espnow.ESPNow() # Instancia de ESP-NOW
e.active(True) # Habilita ESP-NOW

# Direcciones MAC de los dispositivos pares (A, C, D, E)
MAC_A = ubinascii.unhexlify('CCDBA797A9E4'.replace(':', ''))
MAC_C = ubinascii.unhexlify('CCDBA796F13C'.replace(':', ''))
MAC_D = ubinascii.unhexlify('C8F09E79669C'.replace(':', ''))
MAC_E = ubinascii.unhexlify('9C9C1FC807C4'.replace(':', ''))

# Agrega pares para comunicación
e.add_peer(MAC_A)
e.add_peer(MAC_C)
e.add_peer(MAC_D)
e.add_peer(MAC_E)

"""Estados del sistema"""
STATE_MONITOR = 0 # Modo monitor (mensajes entrantes)
STATE_MENU = 1 # Menú principal
STATE_SEND_TARGET = 2 # Selección de destino para enviar mensaje
STATE_SEND_MSG = 3 # Selección de mensaje a enviar
STATE_SLEEP_CONFIRM = 4 # Confirmación para deep sleep
STATE_SENSOR_DISPLAY = 5 # Visualización de datos de sensores
current_state = STATE_MONITOR # Estado inicial

"""Variables de control"""
current_option = 0 # Opción seleccionada en menús
last_messages = [] # Historial de últimos mensajes recibidos
selected_target = 0 # Índice del dispositivo destino (0=A, 1=C, etc.)
sensor_data = None # Datos de sensor recibidos (formato S:origen:datos)

"""Mensajes predefinidos"""
MESSAGES = [
    "Te copio, mensaje recibido",
    "Enviame un mensaje",
    "Ya no envíes mensaje",

```



```

    "HOLI xd"
]

"""Funciones de visualizacion"""
def blink_led(led): # Parpadeo breve de un LED
    led.value(1)
    time.sleep(0.2)
    led.value(0)

def clear_display(): # Limpia la pantalla OLED
    oled.fill(0)
    oled.show()

def show_text(line1="", line2="", line3="", line4=""): # Muestra hasta 4 líneas de texto
    en la OLED
    clear_display()
    oled.text(line1, 0, 0)
    oled.text(line2, 0, 16)
    oled.text(line3, 0, 32)
    oled.text(line4, 0, 48)
    oled.show()

def show_monitor(): # Muestra el último mensaje recibido
    if not last_messages:
        show_text("Esperando mensajes...", "Estado: Activo", "", "Menu: Btn1")
        return

    msg, origin, _ = last_messages[-1]
    show_text(
        "Ultimo mensaje:",
        f"De: {origin}",
        "Contenido:",
        msg[:16] + (msg[16:] and '...')
    )

def show_menu(): # Muestra el menú principal
    options = [
        "Menu Principal",
        "> Ver" if current_option == 0 else " Ver",
        "> Enviar" if current_option == 1 else " Enviar",
        "> Dormir" if current_option == 2 else " Dormir",
        "> Sensor" if current_option == 3 else " Sensor"
    ]
    show_text(options[0], options[1], options[2], options[3])

def show_send_target(): # Muestra opciones de destino para enviar mensaje
    targets = ["A", "C", "D", "E"]
    title = "Enviar a:"

    # Determinar qué opciones mostrar basado en current_option
    if current_option < 3: # Mostrar A, C, D
        options = [

```

```

        title,
        f"> {targets[0]}" if current_option == 0 else f" {targets[0]}",
        f"> {targets[1]}" if current_option == 1 else f" {targets[1]}",
        f"> {targets[2]}" if current_option == 2 else f" {targets[2]}"
    ]
else: # Mostrar C, D, E cuando se selecciona E
    options = [
        title,
        f" {targets[1]}",
        f" {targets[2]}",
        f"> {targets[3]}"
    ]

    show_text(options[0], options[1], options[2], options[3])

def show_send_message(): # Muestra mensajes predefinidos para enviar
    target_name = ["A", "C", "D", "E"][selected_target]
    msg_index = current_option % len(MESSAGES)
    show_text(
        f"Enviar a {target_name}:",
        f"> {MESSAGES[msg_index]}",
        "",
        "Ok: Btn2 Sel: Btn3"
    )

def show_sleep_confirm(): # Muestra confirmación para deep sleep
    show_text("Confirmar Deep Sleep", "Presione OK para", "entrar en modo", "sueno profundo")

def show_sensor_data(): # Muestra datos de sensores (ej. acelerómetro)
    if sensor_data:
        try:
            parts = sensor_data.decode().split(':')[2].split(',')
            origin = "A" if sensor_data.startswith(b'S:A') else \
                "C" if sensor_data.startswith(b'S:C') else \
                "D" if sensor_data.startswith(b'S:D') else \
                "E"

            # Encender LED correspondiente
            if origin == "A":
                blink_led(led_a)
            elif origin == "C":
                blink_led(led_c)
            elif origin == "D":
                blink_led(led_d)
            else: # E
                blink_led(led_e)

            if len(parts) == 6:
                show_text(
                    f"Datos de {origin}:",

```

```

        f"AcX:{float(parts[0])/16384:.2f}g",
        f"AcY:{float(parts[1])/16384:.2f}g",
        f"AcZ:{float(parts[2])/16384:.2f}g"
    )
except:
    show_text("Error en datos", "del sensor", "", "")
else:
    show_text("Esperando datos", "del sensor...", "", "Menu: Btn1")

"""Configuracion de wakeup e inicializacion"""
# Configura el pin 12 como fuente de wakeup para deep sleep
esp32.wake_on_ext0(pin=Pin(12, Pin.IN, Pin.PULL_UP), level=esp32.WAKEUP_ALL_LOW)

# Mensaje inicial en la pantalla
show_text("Sistema iniciado", "Esperando mensajes...", "Estado: Activo", "Menu: Btn1")

"""Bucle principal"""
while True:
    try:
        # 1. Recepción de mensajes ESP-NOW (normales o de sensores)
        sender, msg = e.recv(0)
        if msg:
            if msg.startswith(b'S:'): # Datos de sensor
                sensor_data = msg
                if current_state == STATE_MONITOR or current_state ==
STATE_SENSOR_DISPLAY:
                    show_sensor_data()
            else: # Mensaje normal
                origin = "A" if sender == MAC_A else \
                    "C" if sender == MAC_C else \
                    "D" if sender == MAC_D else \
                    "E"

                last_messages.append((msg.decode(), origin, time.time()))

        # Encender LED correspondiente
        if origin == "A":
            blink_led(led_a)
        elif origin == "C":
            blink_led(led_c)
        elif origin == "D":
            blink_led(led_d)
        else: # E
            blink_led(led_e)

        if len(last_messages) > 3:

```

```

        last_messages.pop(0)
    if current_state == STATE_MONITOR:
        show_monitor()

# 2. Manejo de botones y transiciones entre estados
if btn_menu.pressed():
    if current_state == STATE_MONITOR:
        current_state = STATE_MENU
        current_option = 0
        show_menu()
    elif current_state == STATE_MENU:
        current_state = STATE_MONITOR
        show_monitor()
    elif current_state == STATE_SLEEP_CONFIRM:
        current_state = STATE_MENU
        show_menu()
    elif current_state == STATE_SENSOR_DISPLAY:
        current_state = STATE_MONITOR
        show_monitor()
    time.sleep(0.3)

# 3. Ejecución de acciones según el estado actual
if current_state == STATE_MENU:
    if btn_select.pressed():
        current_option = (current_option + 1) % 4
        show_menu()
        time.sleep(0.3)

    if btn_ok.pressed():
        if current_option == 0: # Ver mensajes
            current_state = STATE_MONITOR
            show_monitor()
        elif current_option == 1: # Enviar mensaje
            current_state = STATE_SEND_TARGET
            current_option = 0
            show_send_target()
        elif current_option == 2: # Dormir
            current_state = STATE_SLEEP_CONFIRM
            show_sleep_confirm()
        else: # Ver datos de sensor
            current_state = STATE_SENSOR_DISPLAY
            show_sensor_data()
        time.sleep(0.3)

elif current_state == STATE_SEND_TARGET:
    if btn_select.pressed():
        current_option = (current_option + 1) % 4 # Ahora 4 opciones (A,C,D,E)
        show_send_target()
        time.sleep(0.3)

    if btn_ok.pressed():
        selected_target = current_option

```

```

        current_state = STATE_SEND_MSG
        current_option = 0
        show_send_message()
        time.sleep(0.3)

    elif current_state == STATE_SEND_MSG:
        if btn_select.pressed():
            current_option = (current_option + 1) % len(MESSAGES)
            show_send_message()
            time.sleep(0.3)

        if btn_ok.pressed():
            # Selección del target basado en la opción seleccionada
            target = [MAC_A, MAC_C, MAC_D, MAC_E][selected_target]
            target_name = ["A", "C", "D", "E"][selected_target]
            e.send(target, f"B:{MESSAGES[current_option % len(MESSAGES)]}")
            show_text("Mensaje enviado!", f"A {target_name}:",
MESSAGES[current_option][:16], "")
            time.sleep(2)
            current_state = STATE_MONITOR
            show_monitor()
            time.sleep(0.3)

    elif current_state == STATE_SLEEP_CONFIRM:
        if btn_ok.pressed():
            show_text("Entrando en", "Deep Sleep...", "Despertar con", "Btn OK")
            time.sleep(2)
            deepsleep()
            time.sleep(0.3)

    # 4. Manejo de errores (reinicio si falla)
    elif current_state == STATE_SENSOR_DISPLAY:
        # No se necesita acción adicional, solo mostrar datos
        pass

except Exception as e:
    print("Error:", e)
    time.sleep(1)
    machine.reset()

```

Código: ProtNowABCDEv4_ESP32-ACDE

"""

Alumno: José Joel Aguilar de la Cruz

Carrera: Ing. Electrónica

Matricula: 2173049871

Programa: ProtNowABCDEv4_ESP32-ACDE

Descripcion: El programa permite poder comunicarse
con el ESP32-B (nodo central) mediante el protocolo ESP-NOW
para enviar mensajes y datos de sensor (MPU6050). Este programa
puede ser aplicado para cualquiera de los nodos habidos (A, C, D, E)
y en caso que se desee, se podria agregar algun otro nodo.

Proyecto terminal:

Proyecto Terminal IoT ESP32:
Red de comunicación y monitoreo mediante sensores (MPU6050)

UAMI, CDMX mayo 2025

"""

"""Librerías importadas y sus propósitos"""

```
from machine import Pin, SoftI2C, deepsleep, RTC # Control de hardware: GPIO, I2C, deep
sleep y reloj
import network # Para configuración de red WiFi
import espnow # Protocolo de comunicación peer-to-peer para ESP32
import ssd1306 # Control de pantallas OLED I2C
import time # Funciones de tiempo y delays
import ubinascii # Conversión de datos binarios/hexadecimales
import esp32 # Funciones específicas del ESP32
import math # Funciones matemáticas (sqrt, etc.)
from mpu6050 import MPU6050 # Driver para el sensor MPU6050 (acelerómetro/giroscopio)
```

"""Configuración OLED"""

```
# Configura interfaz I2C para la pantalla OLED
i2c = SoftI2C(scl=Pin(22), sda=Pin(21))
oled = ssd1306.SSD1306_I2C(128, 64, i2c)
```

"""Configuración MPU6050"""

```
mpu = MPU6050(i2c) # Inicializa el sensor MPU6050
MOVE_THRESHOLD = 0.2 # Umbral para detectar movimiento (en g)
CALIBRATION_SAMPLES = 50 # Número de muestras para calibración inicial
gravity_threshold = 0.7 # Umbral para detección de orientación
```

"""Variables para auto-detección de orientación"""

```
orientation = None # Almacena la orientación detectada
calibration_done = False # Bandera de calibración completada
```

def detect_orientation():

"""Detecta la orientación del dispositivo basado en la gravedad."""

global orientation, calibration_done

samples = []

Toma múltiples muestras para promediar

for _ in range(CALIBRATION_SAMPLES):

```

    vals = mpu.get_values()
    samples.append((vals['AcX'], vals['AcY'], vals['AcZ']))
    time.sleep_ms(10)

# Calcula promedios de los ejes
avg_acx = sum(s[0] for s in samples) / CALIBRATION_SAMPLES
avg_acy = sum(s[1] for s in samples) / CALIBRATION_SAMPLES
avg_acz = sum(s[2] for s in samples) / CALIBRATION_SAMPLES

# Convierte a valores de gravedad (1g = 16384)
gx = abs(avg_acx / 16384.0)
gy = abs(avg_acy / 16384.0)
gz = abs(avg_acz / 16384.0)

# Determina la orientación basado en qué eje tiene mayor gravedad
if gz > gravity_threshold and gz > gx and gz > gy:
    orientation = "HORIZONTAL_Z_UP"
elif gy > gravity_threshold and gy > gx and gy > gz:
    orientation = "VERTICAL_Y_UP"
elif gx > gravity_threshold and gx > gy and gx > gz:
    orientation = "VERTICAL_X_UP"
else:
    orientation = "DEFAULT_VERTICAL"

calibration_done = True
update_display("Orientación detectada:", orientation, "", "")
time.sleep(2)

def adjust_axes(acx, acy, acz):
    """Ajusta los ejes según la orientación detectada."""
    if not calibration_done:
        return acx, acy, acz

    # Reasigna ejes según la orientación
    if orientation == "HORIZONTAL_Z_UP":
        return acy, acz, acx
    elif orientation == "VERTICAL_Y_UP":
        return acx, acy, acz
    elif orientation == "VERTICAL_X_UP":
        return acz, acx, acy
    else:
        return acx, acy, acz

def detect_movement(acx, acy, acz):
    """Detecta movimiento basado en la magnitud del vector aceleración."""
    acx_adj, acy_adj, acz_adj = adjust_axes(acx, acy, acz)
    acx_g = acx_adj / 16384.0
    acy_g = acy_adj / 16384.0
    acz_g = acz_adj / 16384.0
    magnitud = math.sqrt(acx_g**2 + acy_g**2 + acz_g**2)
    return abs(magnitud - 1.0) > MOVE_THRESHOLD

```

```

"""Configuración de botones"""
class Button:
    """Clase para manejo de botones con debounce."""
    def __init__(self, pin):
        self.pin = Pin(pin, Pin.IN, Pin.PULL_UP)
        self.last_press = 0
        self.debounce = 300 # Tiempo de debounce en ms

    def pressed(self):
        """Verifica si el botón fue presionado, con debounce."""
        if self.pin.value() == 0 and (time.ticks_ms() - self.last_press) > self.debounce:
            self.last_press = time.ticks_ms()
            return True
        return False

"""Inicializa botones (pines pueden variar según ESP32 específico)"""
btn_send = Button(12) # Botón para enviar/confirmar
btn_select = Button(14) # Botón para seleccionar
btn_menu = Button(13) # Botón para menú

"""Configuración ESP-NOW"""
sta = network.WLAN(network.STA_IF)
sta.active(True)
e = espnow.ESPNow() # Objeto para comunicación ESP-NOW
e.active(True)

"""Configuración de peer (MAC debe ser actualizada según dispositivo B específico)"""
# NOTA: Esta MAC debe coincidir con la del dispositivo ESP32_B en la red
MAC_B = ubinascii.unhexlify('CCDBA7510E44'.replace(':', ''))
e.add_peer(MAC_B)

"""Mensajes disponibles"""
# NOTA: Los mensajes pueden personalizarse para cada ESP32 (A, C, D, E)
MESSAGES = ["Hola", "Mensaje recibido", "No he recibido", "Reintenta"]
current_msg = 0 # Índice del mensaje actual

"""Estados del sistema"""
STATE_NORMAL = 0 # Estado normal (envío de mensajes)
STATE_MENU = 1 # Estado menú (opciones)
STATE_SENSOR = 2 # Estado sensor (envío datos MPU6050)
current_state = STATE_NORMAL
menu_option = 0 # Opción seleccionada en menú
last_send_time = 0 # Último tiempo de envío
SEND_INTERVAL = 100 # Intervalo entre envíos (ms)

def update_display(line1="", line2="", line3="", line4=""):
    """Actualiza la pantalla OLED con hasta 4 líneas de texto."""
    oled.fill(0)
    oled.text(line1, 0, 0)
    oled.text(line2, 0, 16)
    oled.text(line3, 0, 32)
    oled.text(line4, 0, 48)

```



```

oled.show()

def show_menu():
    """Muestra el menú principal en la pantalla OLED."""
    options = [
        "> Enviar mensaje" if menu_option == 0 else "  Enviar mensaje",
        "> Dormir" if menu_option == 1 else "  Dormir",
        "> Sensor" if menu_option == 2 else "  Sensor",
        "OK:Btn2 Sel:Btn3"
    ]
    update_display("Menu Principal", options[0], options[1], options[2])

"""Configuración wakeup para deep sleep (pin puede variar según ESP32)"""
esp32.wake_on_ext0(pin=Pin(12, Pin.IN, Pin.PULL_UP), level=esp32.WAKEUP_ALL_LOW)

"""Mensaje inicial (personalizable para cada ESP32: A, C, D, E)"""
update_display("ESP32_X Lista", "MAC: "+ubinascii.hexlify(sta.config('mac')).decode()[-6:],
"Menu: Btn1", "Sel:Btn3 OK:Btn2")

"""Bucle principal"""
while True:
    # Manejo del botón menú
    if btn_menu.pressed():
        if current_state == STATE_NORMAL:
            current_state = STATE_MENU
            menu_option = 0
            show_menu()
        elif current_state == STATE_SENSOR:
            current_state = STATE_NORMAL
            update_display("ESP32_X Lista", f"Msg: {MESSAGES[current_msg]}", "Menu: Btn1",
"Sel:Btn3 OK:Btn2")
        else:
            current_state = STATE_NORMAL
            update_display("ESP32_X Lista", f"Msg: {MESSAGES[current_msg]}", "Menu: Btn1",
"Sel:Btn3 OK:Btn2")
            time.sleep(0.3)

    # Estado normal (envío de mensajes)
    if current_state == STATE_NORMAL:
        if btn_select.pressed():
            current_msg = (current_msg + 1) % len(MESSAGES)
            update_display("Mensaje selec:", MESSAGES[current_msg], "", "")
            time.sleep(0.5)
            update_display("ESP32_X Lista", f"Msg: {MESSAGES[current_msg]}", "Menu: Btn1",
"Sel:Btn3 OK:Btn2")

        if btn_send.pressed():
            # NOTA: El prefijo del mensaje (A:, C:, D:, E:) debe coincidir con el ESP32
            # específico
            msg = f"X:{MESSAGES[current_msg]}"
            e.send(MAC_B, msg)
            update_display("Enviando a B:", msg[2:16], "", "...")

```

```

        time.sleep(1)
        update_display("ESP32_X Lista", f"Ultimo envio:", msg[2:16], "")

# Estado menú
elif current_state == STATE_MENU:
    if btn_select.pressed():
        menu_option = (menu_option + 1) % 3
        show_menu()
        time.sleep(0.3)

    if btn_send.pressed():
        if menu_option == 0:
            current_state = STATE_NORMAL
            update_display("ESP32_X Lista", f"Msg: {MESSAGES[current_msg]}", "Menu:
Btn1", "Sel:Btn3 OK:Btn2")
        elif menu_option == 1:
            update_display("Entrando en", "Deep Sleep...", "Despertar con", "Btn
Confirmar")
            time.sleep(2)
            deepsleep()
        else:
            current_state = STATE_SENSOR
            calibration_done = False
            update_display("Modo Sensor", "Calibrando...", "", "")
            time.sleep(0.3)

# Estado sensor (envío datos MPU6050)
elif current_state == STATE_SENSOR:
    if not calibration_done:
        detect_orientation()

    now = time.time()
    if time.time_diff(now, last_send_time) > SEND_INTERVAL:
        vals = mpu.get_values()
        if detect_movement(vals["AcX"], vals["AcY"], vals["AcZ"]):
            acx_adj, acy_adj, acz_adj = adjust_axes(vals['AcX'], vals['AcY'],
vals['AcZ'])
            # NOTA: El prefijo del mensaje (S:A:, S:C:, etc.) debe coincidir con el
ESP32 específico
            msg =
f"S:X:{vals['AcX']},{vals['AcY']},{vals['AcZ']},{vals['GyX']},{vals['GyY']},{vals['GyZ']}"
            e.send(MAC_B, msg)
            last_send_time = now
            update_display("Enviando datos:",
                "AcX: {:.2f}g".format(acx_adj/16384),
                "AcY: {:.2f}g".format(acy_adj/16384),
                "AcZ: {:.2f}g".format(acz_adj/16384))

# Recepción de mensajes
sender, msg = e.recv(0)
if msg:

```

```

        update_display("Recibido:", msg.decode()[2:16], "De:
"+ubinascii.hexlify(sender).decode()[-6:], "")
        time.sleep(2)
        if current_state == STATE_SENSOR:
            update_display("Modo Sensor", "Enviando datos", "a ESP32_B", "Menu: Btn1")
        else:
            update_display("ESP32_X Lista", f"Msg: {MESSAGES[current_msg]}", "Menu: Btn1",
"Sel:Btn3 OK:Btn2")

```

Código: ProtNowABCDEv4 ESP32-ACDE Telegram

"""

Alumno: José Joel Aguilar de la Cruz

Carrera: Ing. Electrónica

Matricula: 2173049871

Programa:ProtNowABCDEv4_ESP32-ACDE_Telegram

Descripcion: El programa permite poder comunicarse con el ESP32-B (nodo central) mediante el protocolo ESP-NOW para enviar mensajes y datos de sensor (MPU6050). Este programa puede ser aplicado para cualquiera de los nodos habidos (A, C, D, E) y en caso que se desee, se podria agregar algun otro nodo. Esta version es para trabajar correctamente con el servidor de Telegram, permitiendo enviar mensajes y datos inerciales.

Proyecto terminal:

Proyecto Terminal IoT ESP32:

Red de comunicación y monitoreo mediante sensores (MPU6050)

UAMI, CDMX mayo 2025

"""

"""Librerías importadas y sus propósitos"""

from machine import Pin, SoftI2C, deepsleep, RTC # Control de hardware: GPIO, I2C, deep sleep y reloj

import network # Para configuración de red WiFi

import espnow # Protocolo de comunicación peer-to-peer para ESP32

import ssd1306 # Control de pantallas OLED I2C

import time # Funciones de tiempo y delays

import ubinascii # Conversión de datos binarios/hexadecimales

import esp32 # Funciones específicas del ESP32

import math # Funciones matemáticas (sqrt, etc.)

from mpu6050 import MPU6050 # Driver para el sensor MPU6050 (acelerómetro/giroscopio)

"""Configuración OLED"""

Configura interfaz I2C para la pantalla OLED

i2c = SoftI2C(scl=Pin(22), sda=Pin(21))

oled = ssd1306.SSD1306_I2C(128, 64, i2c)

"""Configuración MPU6050"""

mpu = MPU6050(i2c) # Inicializa el sensor MPU6050

```

MOVE_THRESHOLD = 0.2 # Umbral para detectar movimiento (en g)
CALIBRATION_SAMPLES = 50 # Número de muestras para calibración inicial
gravity_threshold = 0.7 # Umbral para detección de orientación

"""Variables para auto-detección de orientación"""
orientation = None # Almacena la orientación detectada
calibration_done = False # Bandera de calibración completada

"""Configuración WIFI"""
WIFI_SSID = "Tu_SSID" # Reemplaza con tu SSID de WiFi
WIFI_PASSWORD = "Tu_Contraseña" # Reemplaza con tu contraseña de WiFi

def detect_orientation():
    """Detecta la orientación del dispositivo basado en la gravedad."""
    global orientation, calibration_done
    samples = []
    # Toma múltiples muestras para promediar
    for _ in range(CALIBRATION_SAMPLES):
        vals = mpu.get_values()
        samples.append((vals['AcX'], vals['AcY'], vals['AcZ']))
        time.sleep_ms(10)

    # Calcula promedios de los ejes
    avg_acx = sum(s[0] for s in samples) / CALIBRATION_SAMPLES
    avg_acy = sum(s[1] for s in samples) / CALIBRATION_SAMPLES
    avg_acz = sum(s[2] for s in samples) / CALIBRATION_SAMPLES

    # Convierte a valores de gravedad (1g = 16384)
    gx = abs(avg_acx / 16384.0)
    gy = abs(avg_acy / 16384.0)
    gz = abs(avg_acz / 16384.0)

    # Determina la orientación basado en qué eje tiene mayor gravedad
    if gz > gravity_threshold and gz > gx and gz > gy:
        orientation = "HORIZONTAL_Z_UP"
    elif gy > gravity_threshold and gy > gx and gy > gz:
        orientation = "VERTICAL_Y_UP"
    elif gx > gravity_threshold and gx > gy and gx > gz:
        orientation = "VERTICAL_X_UP"
    else:
        orientation = "DEFAULT_VERTICAL"

    calibration_done = True
    update_display("Orientacion detectada:", orientation, "", "")
    time.sleep(2)

def adjust_axes(acx, acy, acz):
    """Ajusta los ejes según la orientación detectada."""
    if not calibration_done:
        return acx, acy, acz

    # Reasigna ejes según la orientación

```

```

    if orientation == "HORIZONTAL_Z_UP":
        return acy, acz, acx
    elif orientation == "VERTICAL_Y_UP":
        return acx, acy, acz
    elif orientation == "VERTICAL_X_UP":
        return acz, acx, acy
    else:
        return acx, acy, acz

def detect_movement(acx, acy, acz):
    """Detecta movimiento basado en la magnitud del vector aceleración."""
    acx_adj, acy_adj, acz_adj = adjust_axes(acx, acy, acz)
    acx_g = acx_adj / 16384.0
    acy_g = acy_adj / 16384.0
    acz_g = acz_adj / 16384.0
    magnitude = math.sqrt(acx_g**2 + acy_g**2 + acz_g**2)
    return abs(magnitude - 1.0) > MOVE_THRESHOLD

"""Configuración de botones"""
class Button:
    """Clase para manejo de botones con debounce."""
    def __init__(self, pin):
        self.pin = Pin(pin, Pin.IN, Pin.PULL_UP)
        self.last_press = 0
        self.debounce = 300 # Tiempo de debounce en ms

    def pressed(self):
        """Verifica si el botón fue presionado, con debounce."""
        if self.pin.value() == 0 and (time.ticks_ms() - self.last_press) > self.debounce:
            self.last_press = time.ticks_ms()
            return True
        return False

"""Inicializa botones (pines pueden variar según ESP32 específico)"""
btn_send = Button(12) # Botón para enviar/confirmar
btn_select = Button(14) # Botón para seleccionar
btn_menu = Button(13) # Botón para menú

"""Configuración ESP-NOW"""
sta = network.WLAN(network.STA_IF)
sta.active(True)
sta.connect(WIFI_SSID, WIFI_PASSWORD) # Conectar a la red WiFi
e = espnow.ESPNow() # Objeto para comunicación ESP-NOW
e.active(True)

"""Configuración de peer (MAC debe ser actualizada según dispositivo B específico)"""
# NOTA: Esta MAC debe coincidir con la del dispositivo ESP32_B en la red
MAC_B = ubinascii.unhexlify('CCDBA7510E44'.replace(':', ''))
e.add_peer(MAC_B)

"""Mensajes disponibles"""
# NOTA: Los mensajes pueden personalizarse para cada ESP32 (A, C, D, E)

```

```

MESSAGES = ["Hola", "Mensaje recibido", "No he recibido", "Reintenta"]
current_msg = 0 # Índice del mensaje actual

"""Estados del sistema"""
STATE_NORMAL = 0 # Estado normal (envío de mensajes)
STATE_MENU = 1 # Estado menú (opciones)
STATE_SENSOR = 2 # Estado sensor (envío datos MPU6050)
current_state = STATE_NORMAL
menu_option = 0 # Opción seleccionada en menú
last_send_time = 0 # Último tiempo de envío
SEND_INTERVAL = 100 # Intervalo entre envíos (ms)

def update_display(line1="", line2="", line3="", line4=""):
    """Actualiza la pantalla OLED con hasta 4 líneas de texto."""
    oled.fill(0)
    oled.text(line1, 0, 0)
    oled.text(line2, 0, 16)
    oled.text(line3, 0, 32)
    oled.text(line4, 0, 48)
    oled.show()

def show_menu():
    """Muestra el menú principal en la pantalla OLED."""
    options = [
        "> Enviar mensaje" if menu_option == 0 else " Enviar mensaje",
        "> Dormir" if menu_option == 1 else " Dormir",
        "> Sensor" if menu_option == 2 else " Sensor",
        "OK:Btn2 Sel:Btn3"
    ]
    update_display("Menu Principal", options[0], options[1], options[2])

"""Configuración wakeup para deep sleep (pin puede variar según ESP32)"""
esp32.wake_on_ext0(pin=Pin(12, Pin.IN, Pin.PULL_UP), level=esp32.WAKEUP_ALL_LOW)

"""Mensaje inicial (personalizable para cada ESP32: A, C, D, E)"""
update_display("ESP32_X Lista", "MAC: "+ubinascii.hexlify(sta.config('mac')).decode()[-6:],
"Menu: Btn1", "Sel:Btn3 OK:Btn2")

"""Bucle principal"""
while True:
    # Manejo del botón menú
    if btn_menu.pressed():
        if current_state == STATE_NORMAL:
            current_state = STATE_MENU
            menu_option = 0
            show_menu()
        elif current_state == STATE_SENSOR:
            current_state = STATE_NORMAL
            update_display("ESP32_X Lista", f"Msg: {MESSAGES[current_msg]}", "Menu: Btn1",
"Sel:Btn3 OK:Btn2")
        else:
            current_state = STATE_NORMAL

```

```

        update_display("ESP32_X Lista", f"Msg: {MESSAGES[current_msg]}", "Menu: Btn1",
"Sel:Btn3 OK:Btn2")
        time.sleep(0.3)

# Estado normal (envío de mensajes)
if current_state == STATE_NORMAL:
    if btn_select.pressed():
        current_msg = (current_msg + 1) % len(MESSAGES)
        update_display("Mensaje selec:", MESSAGES[current_msg], "", "")
        time.sleep(0.5)
        update_display("ESP32_X Lista", f"Msg: {MESSAGES[current_msg]}", "Menu: Btn1",
"Sel:Btn3 OK:Btn2")

    if btn_send.pressed():
        # NOTA: El prefijo del mensaje (A:, C:, D:, E:) debe coincidir con el ESP32
específico
        msg = f"X:{MESSAGES[current_msg]}"
        e.send(MAC_B, msg)
        update_display("Enviando a B:", msg[2:16], "", "...")
        time.sleep(1)
        update_display("ESP32_X Lista", f"Ultimo envio:", msg[2:16], "")

# Estado menú
elif current_state == STATE_MENU:
    if btn_select.pressed():
        menu_option = (menu_option + 1) % 3
        show_menu()
        time.sleep(0.3)

    if btn_send.pressed():
        if menu_option == 0:
            current_state = STATE_NORMAL
            update_display("ESP32_X Lista", f"Msg: {MESSAGES[current_msg]}", "Menu:
Btn1", "Sel:Btn3 OK:Btn2")
        elif menu_option == 1:
            update_display("Entrando en", "Deep Sleep...", "Despertar con", "Btn
Confirmar")

            time.sleep(2)
            deepsleep()
        else:
            current_state = STATE_SENSOR
            calibration_done = False
            update_display("Modo Sensor", "Calibrando...", "", "")
            time.sleep(0.3)

# Estado sensor (envío datos MPU6050)
elif current_state == STATE_SENSOR:
    if not calibration_done:
        detect_orientation()

    now = time.ticks_ms()
    if time.ticks_diff(now, last_send_time) > SEND_INTERVAL:

```

```

        vals = mpu.get_values()
        if detect_movement(vals["AcX"], vals["AcY"], vals["AcZ"]):
            acx_adj, acy_adj, acz_adj = adjust_axes(vals['AcX'], vals['AcY'],
vals['AcZ'])
            # NOTA: El prefijo del mensaje (S:A:, S:C:, etc.) debe coincidir con el
ESP32 específico
            msg =
f"S:X:{vals['AcX']},{vals['AcY']},{vals['AcZ']},{vals['GyX']},{vals['GyY']},{vals['GyZ']}"
            e.send(MAC_B, msg)
            last_send_time = now
            update_display("Enviando datos:",
                           "AcX: {:.2f}g".format(acx_adj/16384),
                           "AcY: {:.2f}g".format(acy_adj/16384),
                           "AcZ: {:.2f}g".format(acz_adj/16384))

# Recepción de mensajes
sender, msg = e.recv(0)
if msg:
    update_display("Recibido:", msg.decode()[2:16], "De:
"+ubinascii.hexlify(sender).decode()[-6:], "")
    time.sleep(2)
    if current_state == STATE_SENSOR:
        update_display("Modo Sensor", "Enviando datos", "a ESP32_B", "Menu: Btn1")
    else:
        update_display("ESP32_X Lista", f"Msg: {MESSAGES[current_msg]}", "Menu: Btn1",
"Sel:Btn3 OK:Btn2")

```

Código: mpu6050.py

```

# mpu6050.py - Librería completa para el sensor MPU6050
from machine import I2C
import ustruct
import time

class MPU6050:
    def __init__(self, i2c, address=0x68):
        self.i2c = i2c
        self.address = address
        self._init_sensor()

    def _write_byte(self, reg, value):
        self.i2c.writeto_mem(self.address, reg, bytearray([value]))

    def _read_byte(self, reg):
        return self.i2c.readfrom_mem(self.address, reg, 1)[0]

    def _read_bytes(self, reg, length=1):
        return self.i2c.readfrom_mem(self.address, reg, length)

```



```

def _init_sensor(self):
    # Wake up the MPU6050 (exit sleep mode)
    self._write_byte(0x6B, 0x00)
    # Configure gyroscope range ( $\pm 250$  °/s)
    self._write_byte(0x1B, 0x00)
    # Configure accelerometer range ( $\pm 2g$ )
    self._write_byte(0x1C, 0x00)
    # Configure low pass filter (5Hz bandwidth)
    self._write_byte(0x1A, 0x06)
    time.sleep_ms(100) # Wait for stabilization

def get_raw_values(self):
    return self._read_bytes(0x3B, 14)

def get_ints(self):
    b = self.get_raw_values()
    return [int(x) for x in b]

def bytes_toint(self, firstbyte, secondbyte):
    value = (firstbyte << 8) | secondbyte
    if value >= 0x8000:
        value = -((65535 - value) + 1)
    return value

def get_values(self):
    raw = self.get_raw_values()
    vals = {}
    vals["AcX"] = self.bytes_toint(raw[0], raw[1])
    vals["AcY"] = self.bytes_toint(raw[2], raw[3])
    vals["AcZ"] = self.bytes_toint(raw[4], raw[5])
    vals["Tmp"] = self.bytes_toint(raw[6], raw[7]) / 340.00 + 36.53
    vals["GyX"] = self.bytes_toint(raw[8], raw[9])
    vals["GyY"] = self.bytes_toint(raw[10], raw[11])
    vals["GyZ"] = self.bytes_toint(raw[12], raw[13])
    return vals

def get_accel(self):
    raw = self.get_raw_values()
    return (
        self.bytes_toint(raw[0], raw[1]),
        self.bytes_toint(raw[2], raw[3]),
        self.bytes_toint(raw[4], raw[5])
    )

def get_gyro(self):
    raw = self.get_raw_values()
    return (
        self.bytes_toint(raw[8], raw[9]),
        self.bytes_toint(raw[10], raw[11]),
        self.bytes_toint(raw[12], raw[13])
    )

```

```

    )

def get_temp(self):
    raw = self.get_raw_values()
    return self.bytes_toint(raw[6], raw[7]) / 340.00 + 36.53

def calibrate(self, samples=100):
    # Calibrate gyroscope (remove offsets)
    gx, gy, gz = 0, 0, 0
    for _ in range(samples):
        x, y, z = self.get_gyro()
        gx += x
        gy += y
        gz += z
        time.sleep_ms(10)

    self.gyro_offset = (gx/samples, gy/samples, gz/samples)
    return self.gyro_offset

```

Código: ssd1306.py

- Librería completa ssd1306 para pantalla OLED 128x64 I2C

```
import framebuffer
```

Register definitions

```

SET_CONTRAST      = const(0x81)
SET_ENTIRE_ON     = const(0xa4)
SET_NORM_INV      = const(0xa6)
SET_DISP         = const(0xae)
SET_MEM_ADDR      = const(0x20)
SET_COL_ADDR      = const(0x21)
SET_PAGE_ADDR     = const(0x22)
SET_DISP_START_LINE = const(0x40)
SET_SEG_REMAP     = const(0xa0)
SET_MUX_RATIO     = const(0xa8)
SET_COM_OUT_DIR   = const(0xc0)
SET_DISP_OFFSET   = const(0xd3)
SET_COM_PIN_CFG   = const(0xda)
SET_DISP_CLK_DIV  = const(0xd5)
SET_PRECHARGE     = const(0xd9)
SET_VCOM_DESEL    = const(0xdb)
SET_CHARGE_PUMP   = const(0x8d)

```

```

class SSD1306_I2C(framebuffer.FrameBuffer):
    def __init__(self, width, height, i2c, addr=0x3c):
        self.i2c = i2c
        self.addr = addr
        self.temp = bytearray(2)
        self.width = width
        self.height = height

```

```

self.pages = self.height // 8
self.buffer = bytearray(self.pages * self.width)
super().__init__(self.buffer, self.width, self.height, framebuf.MONO_VLSB)
self.init_display()

def init_display(self):
    for cmd in (
        SET_DISP | 0x00, # off
        # address setting
        SET_MEM_ADDR, 0x00, # horizontal
        # resolution and layout
        SET_DISP_START_LINE | 0x00,
        SET_SEG_REMAP | 0x01, # column addr 127 mapped to SEG0
        SET_MUX_RATIO, self.height - 1,
        SET_COM_OUT_DIR | 0x08, # scan from COM[N] to COM0
        SET_DISP_OFFSET, 0x00,
        SET_COM_PIN_CFG, 0x02 if self.width > 2 * self.height else 0x12,
        # timing and driving scheme
        SET_DISP_CLK_DIV, 0x80,
        SET_PRECHARGE, 0x22,
        SET_VCOM_DESEL, 0x30, # 0.83*Vcc
        # display
        SET_CONTRAST, 0xff, # maximum
        SET_ENTIRE_ON, # output follows RAM contents
        SET_NORM_INV, # not inverted
        SET_CHARGE_PUMP, 0x14,
        SET_DISP | 0x01): # on
        self.write_cmd(cmd)
    self.fill(0)
    self.show()

def write_cmd(self, cmd):
    self.temp[0] = 0x80 # Co=1, D/C#=0
    self.temp[1] = cmd
    self.i2c.writeto(self.addr, self.temp)

def write_data(self, buf):
    self.temp[0] = self.addr << 1
    self.temp[1] = 0x40 # Co=0, D/C#=1
    self.i2c.start()
    self.i2c.write(self.temp)
    self.i2c.write(buf)
    self.i2c.stop()

def poweroff(self):
    self.write_cmd(SET_DISP | 0x00)

def poweron(self):
    self.write_cmd(SET_DISP | 0x01)

def contrast(self, contrast):
    self.write_cmd(SET_CONTRAST)
    self.write_cmd(contrast)

def invert(self, invert):

```

```

self.write_cmd(SET_NORM_INV | (invert & 1))

def show(self):
    x0 = 0
    x1 = self.width - 1
    if self.width != 128:
        # narrow displays use centered columns
        col_offset = (128 - self.width) // 2
        x0 += col_offset
        x1 += col_offset
    self.write_cmd(SET_COL_ADDR)
    self.write_cmd(x0)
    self.write_cmd(x1)
    self.write_cmd(SET_PAGE_ADDR)
    self.write_cmd(0)
    self.write_cmd(self.pages - 1)
    self.write_data(self.buffer)

```

7.7 Imagen de los puertos existentes del ESP32



Figura 31. Puertos GPIO del ESP32

Referencias

- (1) Ashton, K. (2009). *That 'Internet of Things' Thing*. RFID Journal. <https://www.rfidjournal.com/that-internet-of-things-thing>
- (2) Barr, M., & Massa, A. (2022). *Embedded Systems: Introduction to ARM Cortex-M Microcontrollers*. CreateSpace.
- (2) Espressif Systems. (2023). *ESP32 Technical Reference Manual*. https://www.espressif.com/sites/default/files/documentation/esp32_technical_reference_manual_en.pdf
- (3) Williams, A. (2021). *ESP32 for IoT: A Practical Guide to Programming and Deployment*. Packt Publishing.
- (4) Microchip Technology. (2022). *Comparing ESP32 with Other Microcontrollers in IoT Applications*. <https://www.microchip.com/>
- (5) Santos, J. (2020). *Getting Started with ESP32 and Arduino IDE*. Maker Media.
- (6) Tanenbaum, A. S. (2021). *Computer Networks* (6th ed.). Pearson.
- (7) Espressif Systems. (2023). *ESP-NOW Protocol Documentation*. <https://docs.espressif.com/>
- (8) Espressif Systems. (2023). *ESP-NOW Protocol Guide*. https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-reference/network/esp_now.html
- (9) Zhang, L. (2022). *IoT Communication Protocols: Comparing ESP-NOW, Wi-Fi, and BLE*. IEEE IoT Journal. <https://doi.org/10.1109/JIOT.2022.XXXXXXX>
- (10) Smith, J. (2021). **Practical ESP32: Projects with ESP-NOW and LoRa**. Packt Publishing.
- (11) García, M. (2020). *Wireless Sensor Networks Using ESP-NOW*. Journal of Embedded Systems, *15*(3), 45-60.
- (12) Espressif Systems. (2023). *ESP32 Deep Sleep Mode Technical Reference*. https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-reference/system/deep_sleep.html
- (13) Williams, A. (2022). **Ultra-Low Power IoT with ESP32: Deep Sleep and Power Management**. Packt Publishing.
- (14) Microchip Technology. (2021). *Low-Power Design Techniques for Microcontrollers*. <https://www.microchip.com/>
- (15) Sánchez, J. (2020). *Programación de Sistemas Embebidos con ESP32*. Marcombo.
- (16) Llamas, L. (s.f.). *ESP32 Sleep modes*. Luis Llamas. <https://www.luisllamas.es/esp32-sleep-modes/>

- (17) Espressif Systems. (2023). *I2C Driver API Guide for ESP32*.
<https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-reference/peripherals/i2c.html>
- (18) NXP Semiconductors. (2021). **I2C-bus Specification and User Manual**.
<https://www.nxp.com/>
- (19) Paret, D. (2017). *I2C Bus: From Theory to Practice*. Wiley.
- (20) Texas Instruments. (2020). *Understanding I2C Protocol*. <https://www.ti.com/>
- (21) InvenSense. (2013). **MPU-6050 Product Specification** (Documento DS-000114).
<https://invensense.tdk.com/products/motion-tracking/6-axis/mpu-6050/>
- (22) Premerlani, W., & Bizard, P. (2012). *Direction Cosine Matrix IMU: Theory*.
<https://www.researchgate.net/publication/260273167>
- (23) STMicroelectronics. (2020). **AN4508: Tilt Measurement Using a Low-Accelerometer**.
<https://www.st.com/>
- (24) García, J. (2021). *Sensores Inerciales para IoT: MPU6050 y ESP32*. Revista de Electrónica Aplicada, *15*(2), 45-60.
- (25) InvenSense. (2013). **MPU-6050 Product Specification**. <https://invensense.tdk.com/>
- (26) Premerlani, W., & Bizard, P. (2012). *Direction Cosine Matrix IMU: Theory*. ResearchGate.
- (27) Espressif Systems. (2023). *ESP32 Datasheet*.
https://www.espressif.com/sites/default/files/documentation/esp32_datasheet_en.pdf
- (28) InvenSense. (2013). **MPU-6050 Product Specification**.
<https://invensense.tdk.com/wp-content/uploads/2015/02/MPU-6050-Datasheet1.pdf>
- (29) Smith, J. (2021). *MicroPython for Embedded Systems*. O'Reilly Media.
- (30) García, M. (2020). *Low-Cost Structural Health Monitoring Using MEMS Sensors*. Journal of Civil Engineering, *18*(3), 112-125.
- (31) MicroPython Team. (2022). *MicroPython Documentation*.
<https://docs.micropython.org/>
- (32) Node Telegram Bot API. (s. f.). *npm*. Recuperado en mayo de 2025, de <https://www.npmjs.com/package/node-telegram-bot-api>
- (33) Yagop. (s. f.). *node-telegram-bot-api API documentation*. GitHub. Recuperado en mayo de 2025, de <https://github.com/yagop/node-telegram-bot-api/blob/master/doc/api.md>
- (34) DeepSeek. (2024). *DeepSeek Chat* [Modelo de lenguaje de inteligencia artificial]. DeepSeek. <https://www.deepseek.com>
- (35) OpenAI. (2024). *ChatGPT* (versión GPT-4.5) [Modelo de lenguaje grande].
<https://chat.openai.com/>